

Chapter 6

Contents

Chapters

- 6. Overview of Peripherals & Interfacing
 - 6.1 Introduction to communication
 - 6.2 Introduction to Parallel communication
 - 6.3 Introduction to Serial communication
 - 6.4 RS 232 Standard
 - 6.5 Voltage level converters for RS 232 to TTL, CMOS
 - 6.6 Interfacing Signals
 - 6.7 Memory Interfacing concepts
 - 6.7.1 Memory Segmentation
 - 6.7.2 Memory/Register Banks
 - 6.7.3 Memory Interleaving
 - 6.7.4 Mapping Techniques
 - 6.7.4.1 Memory mapped I/O
 - 6.7.4.2 I/O mapped I/O
 - 6.7.5 Memory Interfacing
 - 6.8 Traditional Peripheral devices
 - 6.8.1 Programmable Peripheral Interface – 8255
 - 6.8.2 USART – 8251
 - 6.8.3 Programmable Interrupt Controller – 8259
 - 6.8.4 Direct Memory Access – 8237

6.1 Introduction to communication

A system is an integration of different components/devices. These different devices are often controlled by a master which is typically a microcontroller. The transfer of data may be initiated by a microcontroller or any device in the system and data may be transferred between any device and a microcontroller or between different devices.

The transmission of data from one device to another other device (or a microcontroller) is known as transmission mode or communication mode. Each data transfer involves a transmitter, receiver and an associated direction. A device that is transmitting or sending the data is known as a transmitter and the one receiving or taking/accepting the data is known as a receiver. The data is sent from transmitter to receiver and hence the direction is toward the receiver (from transmitter). A device that is transmitting the data is considered as an output device and the device accepting or receiving data is considered as an input device. The following are the 3 types of transmission modes and is demonstrated in fig 6.1.

- Simplex
- Half-Duplex
- Full-Duplex

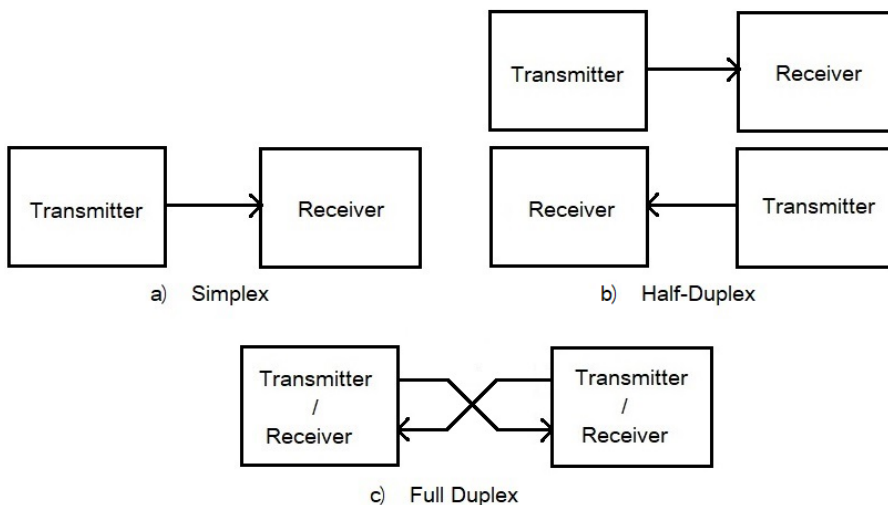


Fig 6.1 Transmission Modes

A transmission mode is called Simplex if a single transmission link exists between the two devices and one device always acts as a transmitter and the other device always acts as a receiver without changing the roles at any point. This is shown in fig 6.1a.

A transmission mode is called Half-Duplex if a single transmission link exists between the two devices, with any one device acting as a transmitter and the other device

acting as a receiver or vice-versa. A change of role may exist in such a transmission mode and only one device can transmit the data at any specific time. This is shown in fig 6.1b.

A transmission mode is called Full-Duplex if two transmission links exist between the two devices, with a transmitter and receiver on two ends of each transmission link. i.e. Device 1 may transmit data on Transmission link1 while the Device 2 acts as a receiver on this link. Similarly, Device 2 may transmit data on Transmission link2 while the Device 1 acts as a receiver on this link. Full-Duplex mode exists when two devices have the capability to transmit and receive at the same time on two different transmission links. Role changing is not required here since both the devices can transmit, receive. This is shown in fig 6.1c.

Most of the current processing and controlling elements are digital or mixed signal devices that utilizes representation of real-world analog signal as a digital value which is a combination of 1's and 0's. These devices are referred to as 8-bit, 16-bit, 32-bit or 64-bit devices (xx-bit) and they operate on that specified number of bits at once as they typically have registers, bus and ALU of that specific size. The devices transfer and operate on that data. It can be summarised that data is transferred by these devices in one of the following 2 ways.

- Parallel Data Transfer
- Serial Data Transfer

In a parallel data transfer mechanism, the device would be able to transfer the given set of data bits (i.e. xx-bit) all together at once on parallel lines or a bus. In a serial data transfer mechanism, the device would be able to transfer the given set of data bits (i.e. xx-bit) serially on a single data line only one bit at a time until the complete transfer of all bits.

Microcontrollers are typically used in the design of embedded systems. They are designed by interfacing microcontroller with different sensors, actuators, or may be with other microcontrollers etc. These different components in an embedded system are often required to communicate or establish connection with each other for data transfer and to perform the required operations. The transfer of data may be preceded and/or followed up by communication signals which are also known as Handshaking signals. The following sections discuss the ways of Parallel, Serial Data Transfer schemes and handshaking signals.

6.2 Introduction to Parallel communication

The parallel data transfer is a mechanism in which the device that is operating on a set of bits (8-Bit, 16-Bit, 31-Bit, 64-Bits etc) is able to transfer all the specified bits at the same time on a set of parallel lines. This group of lines is called as a bus is termed as address bus, data bus or control bus based on the type of information that is carried on the bus. The printer is a good example for device that uses parallel communication.

As discussed earlier the parallel data transfer may be preceded or followed up by control or handshaking signals. Such a transfer of data with or without the involvement of control and handshake signals is considered as parallel communication.

Different control and handshaking signals are provided by different devices. The ALE-Address Latch Enable, RD - Read, WR – Write, IO/M – Input Output/Memory, DEN – Data Enable, DT/R – Data Transmit/Receive, Strobe, Acknowledge, Input Buffer Full, Output Buffer Full etc are some examples of control and handshaking signals. The different types of parallel data transfers employed for Input and Output are:

- Simple I/O
- Simple Strobed I/O
- Single Handshake I/O
- Double Handshake I/O

The parallel data transfer of unidirectional setup is shown in fig 6.2 and a bidirectional setup is shown in fig 6.4. The typical waveforms for different parallel data transfer methods are presented in fig 6.3. Based on the type of device an active high logic or an active low logic may be used to represent the handshaking signals. The waveforms presented here make use of active low logic for strobe and active high logic for acknowledge signals.

Simple I/O: Simple Input-Output

A data transfer scheme is considered as a simple I/O when data transfer takes place without any handshake signals. The transmitter or output device simply transmits the data on the parallel lines/bus if the receiving or input device reads this data. There is no strobe or enable, acknowledge or response signals. The Simple I/O form of parallel data transfer is shown in fig 6.2a and corresponding waveform is shown in fig 6.3a.

Simple Strobed I/O: Simple Strobe Input-Output

A data transfer scheme is considered as a simple strobed I/O when a strobe signal is transmitted before the transfer of actual data. The output device first sends a strobe signal indicating that data is being transmitted on the parallel lines/bus and that the input device must read it. In this transfer type only, the transmitter sends the strobe signal, and the receiver does not acknowledge or respond to the transmitter whether it is ready to accept the data or whether it has read the data. The Strobed I/O form of parallel data transfer is shown in fig 6.2b and the corresponding waveform is shown in fig 6.3b. Here the receiving device may be activated upon a low level, negative edge or a positive edge on the strobe signal.

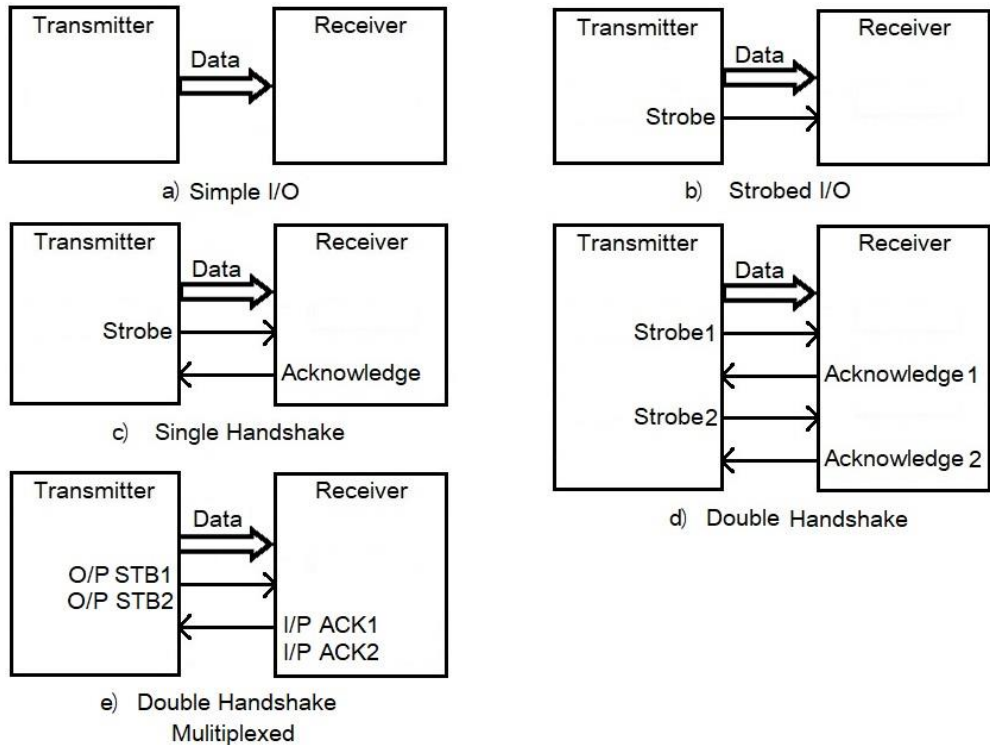


Fig 6.2 Types of Parallel Data Transfer: Unidirectional

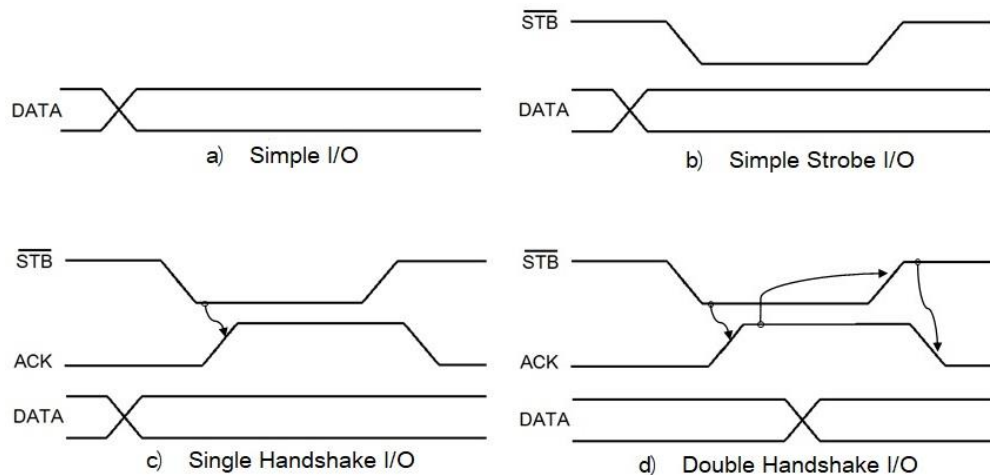


Fig 6.3 Waveforms for different types of Parallel Data Transfer

Single Handshake I/O: Single Handshake Input-Output

A data transfer scheme is considered as a single handshake I/O when a strobe signal is transmitted, and an acknowledgement is received by the by the output device before the transfer of actual data. The output device first sends a strobe signal to query if the receiver

is ready to accept the data. When the receiver is ready to accept data, the receiver generates an acknowledge signal to the transmitter indicating its readiness. The transmitter then transmits the actual data on the parallel lines/bus.

In this transfer type a handshake is formed by the strobe from output device and acknowledge signal from the input device. Hence it is termed as single handshake. The transmitter sends the data only after it receives an acknowledge signal from input device for its earlier generated strobe signal. This transfer type does not ensure that the data is read by the input device as no acknowledge signal is generated after the data is transmitted. The Single handshake I/O form of parallel data transfer is shown in fig 6.2c and the corresponding waveform is shown in fig 6.3c. Here the receiving device may be activated upon a low level, negative edge or a positive edge on the strobe signal.

Double Handshake I/O: Double Handshake Input-Output

A data transfer scheme is considered as a double handshake I/O when a handshake exists before and after the transfer of actual data. The first handshake ensures the readiness of the device to accept the data and the second handshake ensures the reception of the data. The output device first sends a strobe signal to query if the receiver is ready to accept the data. The receiver may be activated upon a -ve edge or low level on strobe line. When the receiver is ready to accept data, the receiver generates an active high acknowledge signal to the transmitter indicating its readiness. This forms the first handshake and the transmitter then transmits the actual data on the parallel lines/bus.

The transmitter sends the data only after it receives an acknowledge signal from input device for its earlier generated strobe signal i.e. the first handshake. Further the output device creates a positive edge followed by high level on the strobe line. The receiver may further be activated upon a +ve edge or high level on strobe line. When the receiver has read the actual data on the parallel lines/bus, it de-asserts the acknowledge line creating a -ve edge on it. This forms the second handshake and indicates the correct transfer of data from output device to the input device. This transfer type also ensures that the data is read by the input device i.e. actual transfer of data from output device to the input device. The first handshake ensures the readiness of input device and second handshake ensures the reception of the data. Since it involves two handshakes it is known as double handshake data transfer.

The second handshake could be created by another strobe and acknowledge signal using separate lines as shown in fig 6.2d or may be conducted on the same strobe and acknowledge lines as shown in fig 6.2e. The above discussion and corresponding waveform shown in fig 6.3d assumes the arrangement shown in fig 6.2e.

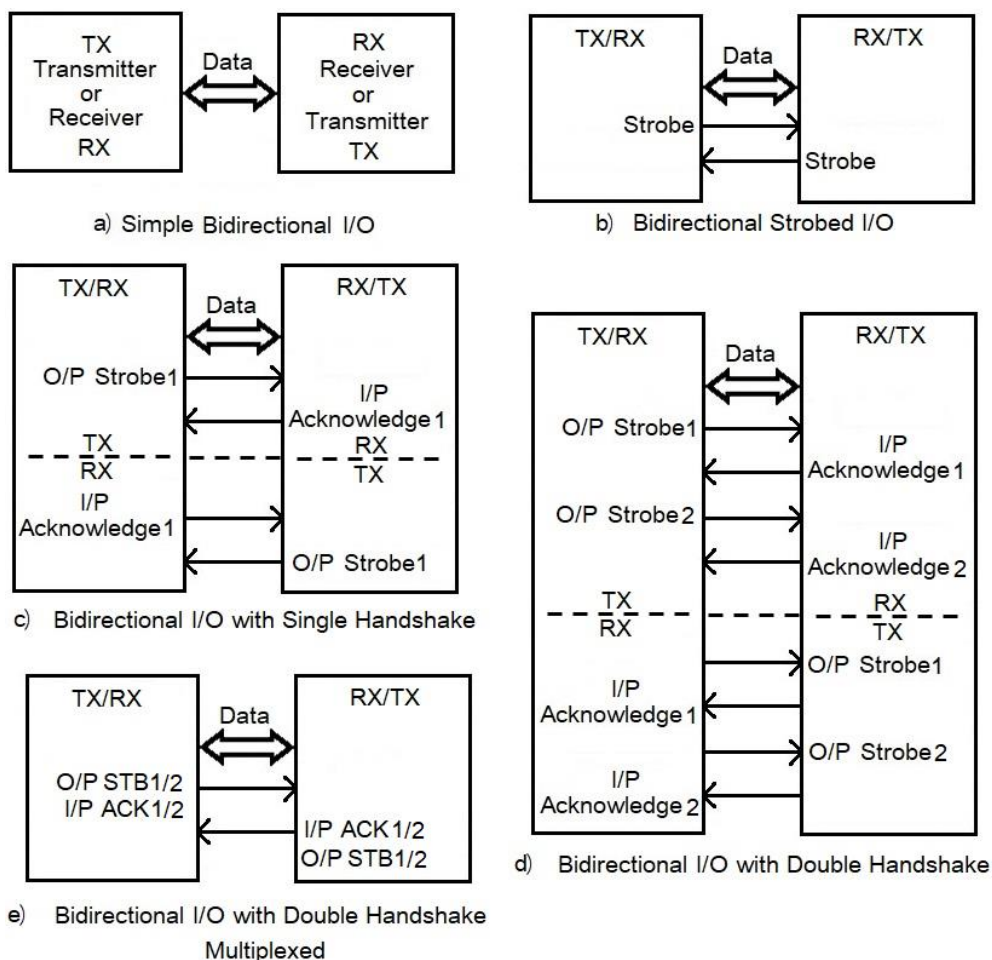


Fig 6.4 Types of Parallel Data Transfer: Bidirectional

All the discussion so far assumes that the data transfer is unidirectional, but the data transfer could be bidirectional and equivalent arrangement of the same is presented in fig 6.4. Also note that the logic levels used may vary based on the devices.

Microprocessors typically do not have the capability to interface different sensors, actuators and other devices directly. Hence a port device such as 8255: PPI – programmable peripheral interface is used for interfacing different peripherals and devices. A typical arrangement of the same is shown in fig 6.5. Details of 8255 are presented in detail later in sec 6.4.

Though microcontrollers generally have many in-built peripherals, all the required peripherals, sensors and actuators may not be built into it. However, they additionally have the capability to interface different sensors, actuators and other peripheral devices directly. Hence a port device such as 8255: PPI – programmable peripheral interface is not required. A typical arrangement of the same is shown in fig 6.6. An 8255 – PPI may be used with

microcontrollers for expanding its capabilities, but they are rarely used since variations of microcontrollers exist with better port capabilities.

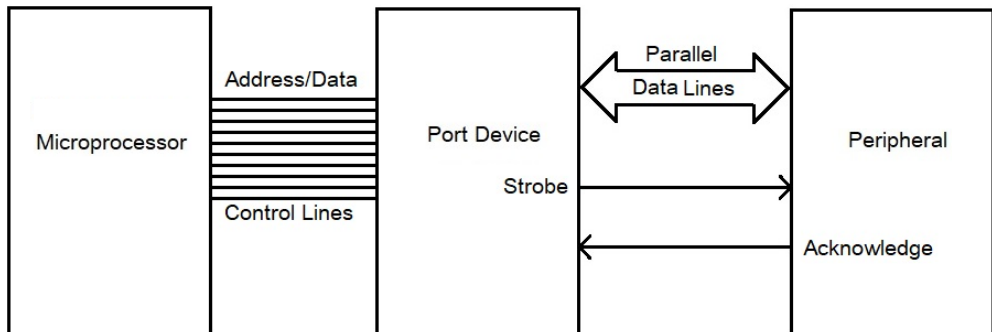


Fig 6.5 Typical Microprocessor Interfacing of Peripherals with Parallel Data Lines

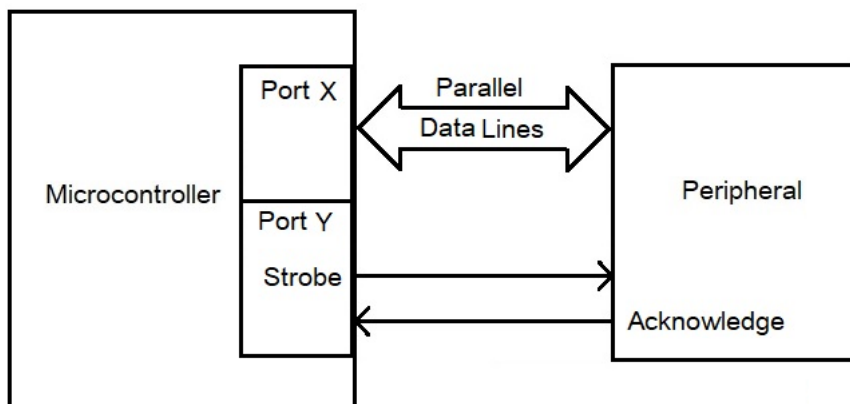


Fig 6.6 Typical Microcontroller Interfacing of Peripherals with Parallel Data Lines

As an example, the two PIC18 devices: PIC18C801, PIC18F8720 has 7 ports that are capable of 8-bit data transfers and hence generally does not require a port device. The PIC18F458 however has only 3 ports that are capable of 8-bit data transfer and hence may require a port device for addition ports or I/Os. In such a case the interfacing circuit would be similar to that of a typical interfacing of microprocessor with the exception that the address, data and control lines originate from the microcontroller ports. In PIC18F458 and PIC18F8720 a similar port device feature is present which is called as parallel slave port or PSP. It is discussed in detail in parallel slave port section of chapter 10.4.

The advantage of parallel communication is that all the bits are transferred at once and is available to the receiver. The disadvantage of parallel communication is that it requires high number of I/O lines which also increases the size of the device. Hence most of the latest devices including the printers are provided with serial interface like USB rather than a parallel interface.

6.3 Introduction to Serial communication

The serial data transfer is a mechanism in which the device that is operating on a set of bits (8-Bit, 16-Bit, 31-Bit, 64-Bits etc) is able to transfer all the specified bits serially on a single line, one bit at a time until the complete transfer of all bits. Since data must be sent bit wise a parallel in serial out shift register is generally employed to generate one data bit per clock by shifting the register.

Numerous serial communication protocols exist which vary based on the type of control and handshaking signals that are used. Some uses mechanism similar to the ones discussed earlier in the parallel communication while some use none or a limited number of handshaking signal lines or varied approach protocols that includes handshaking signals on the data transmission line itself. A detailed description of different serial communication protocols is presented in chapter 10 and 15.

The serial communication can broadly be categorised into two data transfer types or modes. These data transfer modes are as follows.

- Asynchronous
- Synchronous

Asynchronous data transfer:

Asynchronous communication is a type of data transfer mode in which a clock signal is not shared between the two communicating devices. Both the transmitter and receiver may operate on different clock frequencies, but the actual data is transmitted at a specific data rate known as the baud rate. A device that operates using such a signalling is typically called as an UART – Universal Asynchronous Receive Transmit. The different transmission modes of UART are shown in fig 6.7. Typically, UART devices employ full duplex scheme with ground reference shown in fig 6.7f. The various possibilities are presented in fig 6.7a to e.

In asynchronous communication, the data is transmitted as frames and typically involves a set of control signals – the start and stop bits, one signal bit before the start of data transmission and the other after the actual data is transmitted. These start & stop control bits are transmitted on the same line as the data. Some devices also include parity bits along with the start and stop bits whereas some devices provide flexibility for 1 or 2 stop bits and varying number of data bits from 5 to 8. A sample waveform of the same is shown in fig 6.8.

This is an equivalent strobed I/O version of parallel data transfer of fig 6.2b with start and stop bits acting as two strobes on the same line that are generated before and after the transmission of data. Asynchronous communication is preferred when large block of data is not available and only when few bytes or words are to be sent. Typically, systems that immediately process the limited number of available bytes or words received and tolerate

the overhead of start/stop bits etc use this mechanism. It is also used when the data is not continuously available or is arbitrary and when synchronization between transmitter and receiver clock generators or bit synchronization is not required.

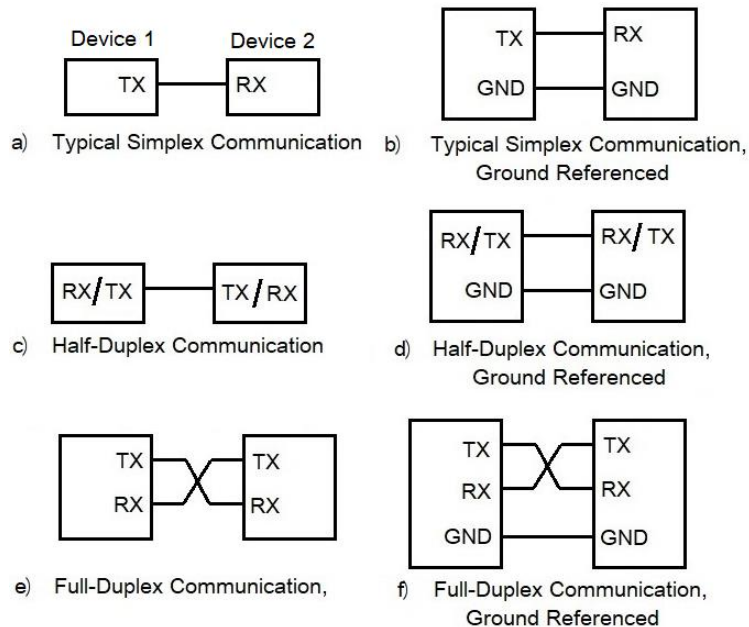


Fig 6.7 Types of UART Transmission Modes

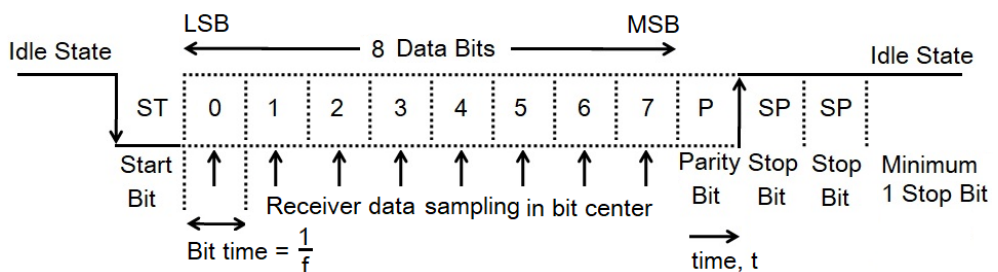


Fig 6.8 Sample UART waveform

The rate at which information is transferred in a communication channel is called as baud rate. The typical Baud rates used are 300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 115200 bits per second. The bit rate and baud rate are identical when the information unit 1 bit is one baud. In the context of serial communication 9600 baud indicates that 9600 bits are being transferred per second. If a baud is considered as a symbol/frame of 10 bits (8 data with 1 start and stop bit) then the baud rate is $9600/10 = 960$. The actual data bits that are transferred in this case are $960 \times 8 = 7680$ bits per second.

Estimation of Overhead:

An 8-bit data with 1 start and stop bit each creates a frame of 10 bits. The actual data is 8 bits, and the extra bits are the 2 bits – start and stop. Hence the overhead is given as the ratio of extra bits to the number of data bits. This gives the overhead as $2/8$ which is equal to 25%. The overhead increases further when parity bit and the second stop is also included. It makes the overhead worse when the number of data bits is reduced. A 5-bit data with 1 start bit, 2 stop bit and 1 parity bit creates a frame of 9 bits where the extra bits are now 4 making the overhead as $4/5$ which is equal to 80%.

Synchronous data transfer:

Synchronous communication is a type of data transfer mode in which a clock signal is shared between the two communicating devices. Typically, the clock is also generated by the transmitter along with the data. The data is sent typically as blocks rather than byte or word etc. This is used for transfer of large data with minimum overhead and when there is a continuous stream of data like video, audio data or accessing large data files etc.

This mechanism allows for configuring the devices to be a master or slave. The master can be a transmitter or receiver, similarly, is the slave. However, the clock is transmitted by the master, and the data transfer is controlled based on this clock. While master is the transmitter, it transmits the clock and sends the data at the respective clock rate. The receiver is enabled by this clock and is expected to read available data at this clock rate.

Typically, a bit is transmitted every clock pulse, and the receiver must read and store the data bit before the next available clock pulse and then group to form byte or word etc from the incoming block data. Similarly, a master can act as a receiver. When the clock is the transmitted by the master the receiver is expected to transfer the data, if it is not ready it transfers IDLE/SYNC characters and when it is ready transfers the data at the clock rate provided by the master.

Since the transmission of data is enabled and controlled through the clock, separate handshaking signal lines or control lines are generally not required for synchronous communication. However, to ensure proper and accurate delivery of data special characters such as SYNC or IDLE characters are used for synchronization and spacing of data blocks in regular intervals. The transmitter sends SYNC characters initially so that the receiver synchronizes with the transmitter before the actual transmission of data blocks. Idle characters are transmitted for inter block gap or when no data is to be sent.

A device that operates using such a signalling typically also includes the features of asynchronous communication and such a device is typically called as an USART – Universal Synchronous Asynchronous Receive Transmit. The most used transmission modes of synchronous communication are shown in fig 6.9. Synchronous devices typically employ half-

duplex transmission mode as shown in fig 6.9a. To reduce noise and offsets the two devices also may share a common ground. Such a representation is shown in fig 6.9b where both the devices common ground referenced. An equivalent full duplex setup may be used when fast exchange of data is required, and it is shown in fig 6.9c and fig 6.9d respectively.

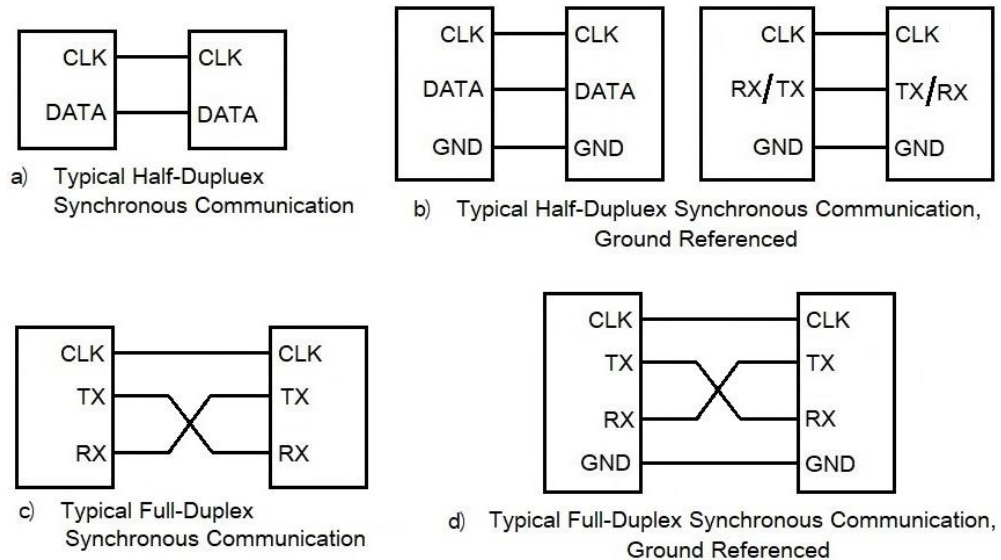


Fig 6.9 Commonly used Synchronous Transmission Modes

The advantage of serial communication is that it requires very few number I/O lines typically 2 lines for bidirectional transfer of data. Serial communication also allows a reduced size of the device as it requires few I/O lines for communication. The disadvantage of serial communication is that the data is sent serially one bit at a time due to which data transfer is slower compared to parallel communication. Additionally, an overhead may exist in some forms of serial communication (ex: UART) to indicate beginning and end of transmitted frame and may also require an inter frame gap. This is required when clock, control or hand shaking signals are not used with serial communication.

6.4 The RS 232 Standard

The earlier and most popular serial communication standard is the RS 232. It is developed as a standard for serial data transfer in communication equipment's specifically for communicating between computer terminals and telephone line models. They typically include computer terminals (Data Terminal Equipment – DTE), modems (Data Communication Equipment - DCE) etc. It is also used in PCs, printers, computer mice and other peripheral devices.

This standard defines interfacing signal's electrical characteristics - voltage levels, signal timing, functionality, connectors pin-out and its physical size for use in equipment's employing the serial communication.

The RS in "RS 232" represents recommended standard. The standard has been revised and upgraded over time as the technology progressed by different associations/institutes such as EIA – Electronic Industries Association, TIA – Telecommunications Industry Association and ANSI – American National Standards Institute. These revisions are termed with unique characters (A to F) and named accordingly as EIA RS-232, EIA RS-232-A, EIA RS-232-B, EIA RS-232-C, EIA-232-D, TIA/EIA-232-E, TIA/EIA-232-F, later ANSI/TIA-232-F-1997 in 2002 and TIA-232-F in 2012. The specifications of TIA/EIA-232-F are given in table 6.1.

Table 6.1: TIA/EIE 232 F specifications (Analog Devices: RS232 Quick Guide)

Specification		Typical values
Mode of Operation		Single-Ended
Number of Drivers & Receivers connected on one line		1 Driver, 1 Receiver
Maximum Cable Length		50 Ft
Maximum Data Rate		20 k Bits/s
Maximum Voltage applied to Driver O/P		$\pm 25V$
Driver Output	Minimum Loaded	$\pm 5V$
	Maximum Loaded	$\pm 15V$
Termination		3K Ω to 7K Ω
Output Slew Rate		30V/ μs (Max)
Receiver Input Voltage range		$\pm 25V$ max
Receiver input sensitivity		$\pm 3V$
Receiver input resistance		3K Ω to 7K Ω

Voltage levels and waveform

The TIA/EIA 232 F specifications are given in table 6.1 which present the maximum voltage levels of transmitter and receiver. Hence the transmitted voltage levels must be in the range of $\pm 15V$, and receiver voltage levels in the range of $\pm 25V$.

A binary 0 is transmitted by a positive voltage in the range of +5V to +15V and is termed as "Space" or "ON". A binary 1 is transmitted by a negative voltage in range of -5V to -15V and is termed as "mark" or "off". The received voltage of binary 0 may be in the range of +3V to +25V and of binary 1 in the range of -3V to -25V. It should be noted that the voltage levels and some specifications vary based on the version of RS 232 standard. The signal voltage level on the line and the actual transmitted data is shown in fig 6.10. a simple representation is shown in fig 6.14. The signal voltage levels and actual data values that are transmitted in asynchronous format of is shown in fig 6.11. The x in the figure indicates that the bit value may be a 1 or 0.

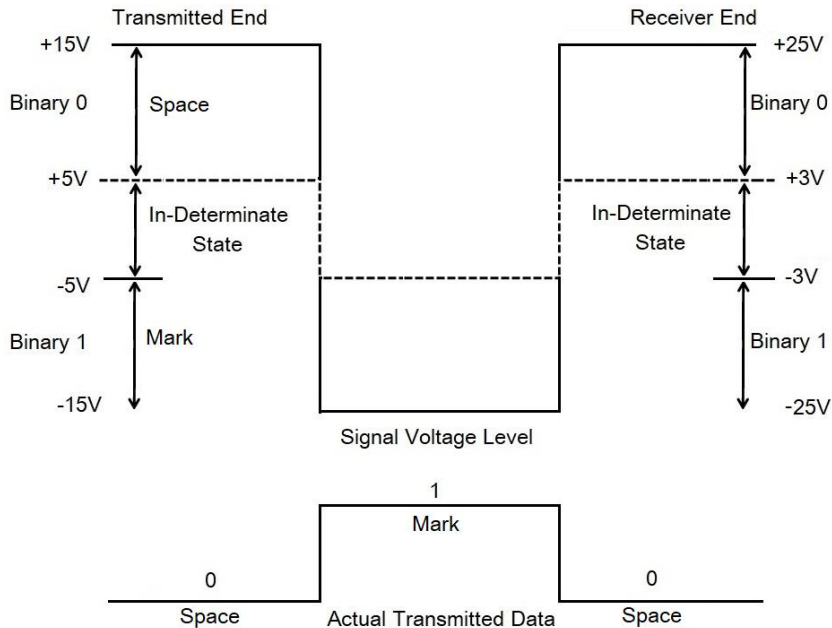


Fig 6.10 Signal voltage level and actual transmitted data

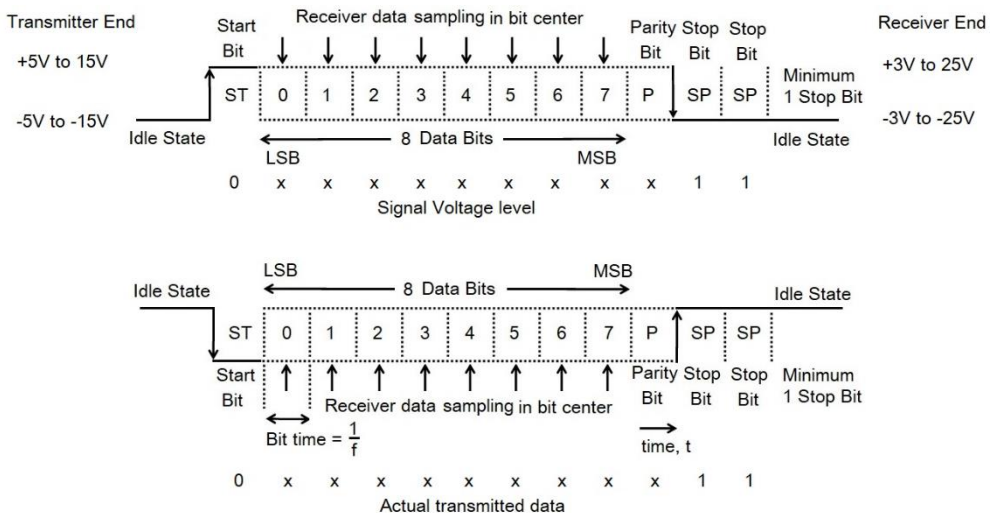


Fig 6.11 Signal voltage & actual transmitted data for asynchronous data frame

Connectors:

Various types of connectors are available, but the RS 232 standard specifies two connectors for serial communication. They are

- DE-9 Connector (commonly called as DB-9 Connector)
- DB-25 Connector

A representation of these two connectors is shown in fig 6.12. The D in the DB/DE represents the D shape of the connector, B or E represents the sub series which corresponds to the shell size. Various sub-series prefixes are A, B, C, D & E of which the E sub series has the smallest footprint. The number 9 or 25 represents the number of pins in the connector.

a) DE-9 connector

The DE9 or the DB9 is a 9-pin connector that is a commonly used connector for asynchronous serial data transmission in PCs due to its small size.

b) DB-25 connector

The DB25 is a 25-pin connector that is used in cases where additional control and handshaking signals are required. Due to its bigger size, it is not commonly used.

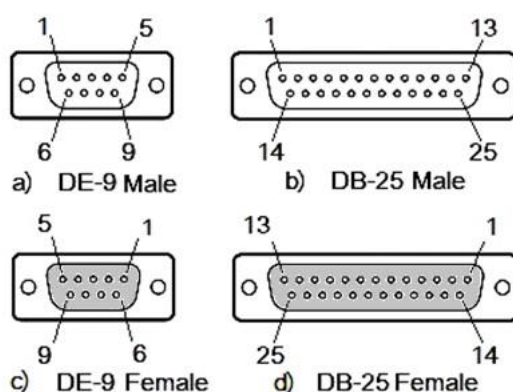


Fig 6.12 Male Connectors specified by RS 232 standard

(Pin Number ordering for male connector is left to right & for female it is right to left)

Table 6.2: DB-9 Connector Signals

Pin No	Signal Name	Abbreviation
1	Carrier Detect/ Data Carrier Detect, Received Line Signal Detect	<u>RLSD</u> (<u>CD</u> , <u>DCD</u>)
2	Receive Data	RD (RXD)
3	Transmit Data	TD (TXD)
4	Data Terminal Ready – DTE Ready	<u>DTR</u>
5	Signal Ground/ Common Return	SGND
6	Data Set Ready – DCE Ready	<u>DCR</u> (<u>DSR</u>)
7	Request to Send	<u>RTS</u>
8	Clear to Send	<u>CTS</u>
9	Ring Indicator	RI

Signals & their functions:

A typical telephone-based communication system typically involves a DTE - Data Terminal Equipment and a DCE - Data Communication Equipment.

a) DTE – Data Terminal Equipment: A DTE is a device that acts a data source, data sink or both. Examples of DTE are terminal, peripheral, computer etc.

b) DCE – Data Communication Equipment: The DCE is a device that establishes, maintain or terminate a connection link for data transmission. It also may perform signal conversion and coding that is required for communication. A typical example of a DCE is a modem.

Each pin of DE-9 or DB-25 is associated with a specific function, reserved or unassigned. The DE-9 connector's 9 pins and the function associated with each pin are presented in Table 6.2. The description of the DE-9 connector pin functions is given below based on typical operation sequence for a type shown in fig 6.13a.

1. **Data terminal ready (DTR):** When a DTE computer or terminal is started; the DTE sends an active low strobe signal to the DCE on this line indicating that it is ready to communication.
2. **Data set ready (DSR):** In response to DTR when the DCE is ready to communicate it sends an active low acknowledge signal on this line to DTE indicating it is connected to the line, it is ready for communication. When telephone connection cannot be made by the modem this signal remains inactive indicating that it is not ready.
3. **Data Carrier Detect (DCD):** When the DCE establishes a connection with the receiver modem and a valid carrier is detected, an active low signal is sent on this line by the DCE to DTE indicating the availability of carrier or receipt of a valid input signal.
4. **Request to send (RTS):** When the DTE is ready to transmit a data byte; DTE sends an active low strobe on this line to DCE indicating its readiness to send data.
5. **Clear to send (CTS):** When the DCE is ready to accept data it sends an active low acknowledgement signal to DTE indicating its readiness to receive the data. This enables the DTE to transmit the data.
6. **Transmit data (TD):** The actual data is transmitted on the TD line from DTE to DCE. It is held in mark state when no data is transmitted.
7. **Received data (RD):** The data from the DCE is received on the RD line by the DTE. It is held in mark state when DCD signal is inactive.
8. **Ring indicator (RI):** When a telephone is ringing a signal is generated by DCE to DTE to indicate the ringing. RI is synchronously turned ON/OFF with the ringing sound. It is used mainly in auto answer system, older modems.
9. **Signal ground (SGND):** This line serves as the common ground connection/ reference for all the signals. This is different from protective ground which is connected to equipment frame that is externally grounded.

The DB25 connector's 25 pins and the function associated with each pin are presented in Table 6.3. The details of DB-25 pins are not discussed in detail. Interested reader may refer to the appropriate documents for their details.

Table 6.3: DB-25 Connector Signals

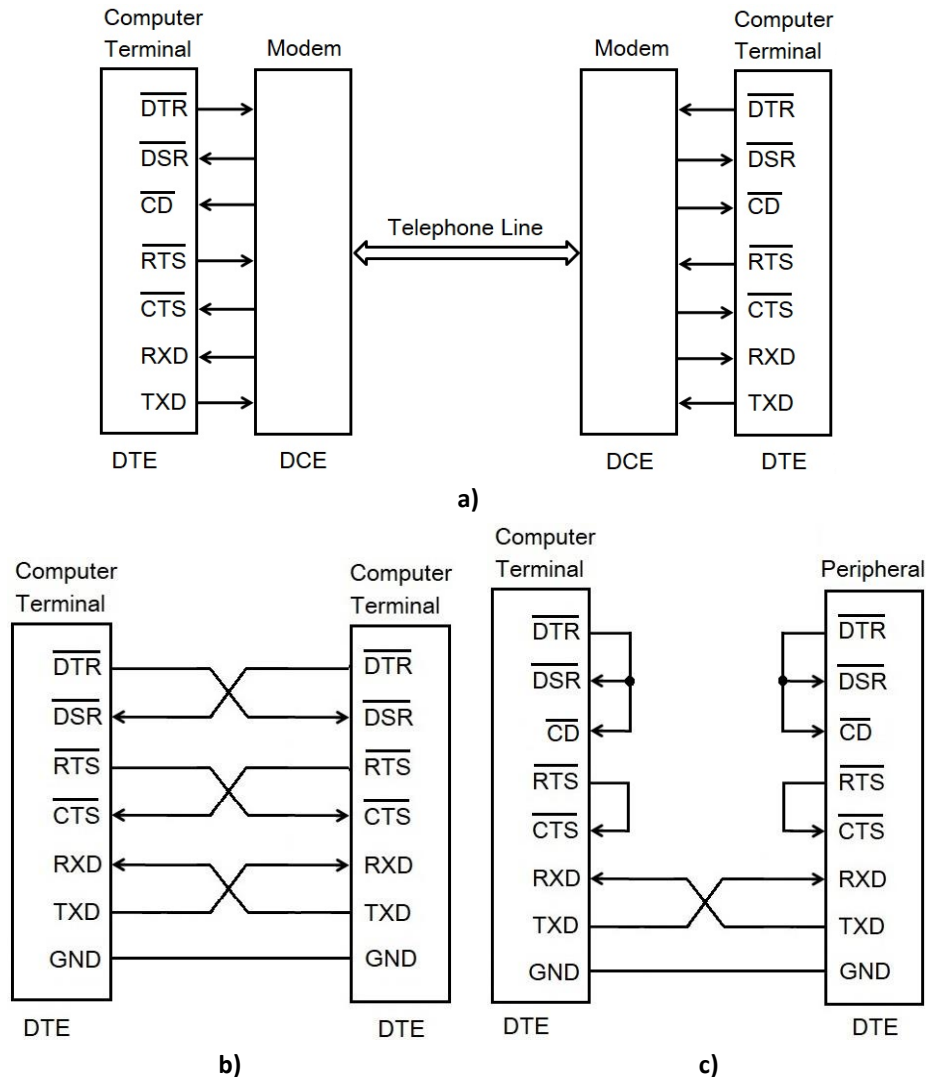
Connector		Signal Name	Abbreviation
DB25	DB9		
1		Shield, Frame/Chassis/Protective Ground	SHD(PGND CGND,FGND)
2	3	Transmit Data	TD (TXD)
3	2	Receive Data	RD (RXD)
4	7	Request to Send	RTS
5	8	Clear to Send	CTS
6	6	Data Set Ready – DCE Ready	DCR(DSR)
7	5	Signal Ground/ Common Return	SGND
8	1	Carrier Detect/ Data Carrier Detect – Received Line Signal Detector	RLSD (CD,DCD)
9		(Reserved) – Testing	
10		(Reserved) – Testing	
11		Unassigned	
12		Secondary - Carrier Detect/ Data Carrier Detect – Received Line Signal Detector or Data Signal Rate Selector (DCE Source)	SRLSD (SCD,SDCD) DSRSC
13		Secondary Clear to Send	SCTS(CTS)
14		Secondary Transmit Data	STD(STXD)
15		Transmitter Clock – TX signal element timing (DCE Source)	TSETC (TC, TXC)
16		Secondary Receive Data	SRD (SRXD)
17		Receiver Clock – RX signal element timing (DCE Source)	RSETC (RC, RXC)
18		Local Loopback	LL
19		Secondary Request to Send	SRTS
20	4	Data Terminal Ready – DTE Ready	DTR
21		Remote Loopback or Signal quality Detector	RL,SQD
22	9	Ring Indicator	RI
23		Data Signal Rate Selector (DTE Source) (DCE Source)	DSRST, DSRSC
24		External Clock or TX Signal element Timing (DTE Source)	TSETT (EXTC)
25		Test Mode – Test Indicator	TM (TI)

Transfer type based on RS 232 Standard:

The signals DTR, DSR, DCD, RTS, and CTS of DE-9 connector are typical examples of handshake signals presented in parallel data transfer types.

The different ways of serial transfers in communication equipment are presented here.

- Data transfer used in system with Modem and Telephone line
- Data transfer used in system with Null Modem: Without Modem
- Simple data transfer without handshake



a) RS 232 connection in system with Modem and Telephone line

b) RS 232 connection in system with Null Modem – Modem without handshake signals

c) RS 232 connection of simple serial port without handshake signals

Fig 6.13 Data Transfer types with RS 232 Standard

a) Data transfer used in system with Modem and Telephone line

A typical operation of such a data transfer system is presented earlier in the process of describing DE-9 connector signals. The function and order of the pins discussed is the operation sequence that is followed in such a system. The actual interface is shown in fig 6.13a.

b) Data transfer used in system with Null Modem

A typical operation of such a data transfer is similar to the earlier presented case type 'a' with the exception that a modem and telephone line is not used. The actual interface is shown in fig 6.13b.

c) Simple data transfer without handshake

A typical operation of such a data transfer does not include any handshake signals and is similar to the type presented in fig 6.7. This is the common serial data transfer type used with PCs. The handshake signals are shorted in this type of transfer and typical serial port interface is shown in fig 6.13c.

The limitation of the RS 232 standard is that it doesn't allow for multi-drop configuration. It only allows for a single driver and receiver. Hence it is superseded by RS 422 and RS 485. The RS 422 allows for single driver with multiple receivers, whereas RS 485 allows for multiple drivers and receivers. Various other limitations exist related to large voltage swing, noise immunity, transfer rate, transmission distance, higher power supply voltages, etc. Other limitations include non-availability of power transmission feature, dual power supply (+ve and -ve), greater size db-25 connector which does not suit the current trend of device miniaturisation.

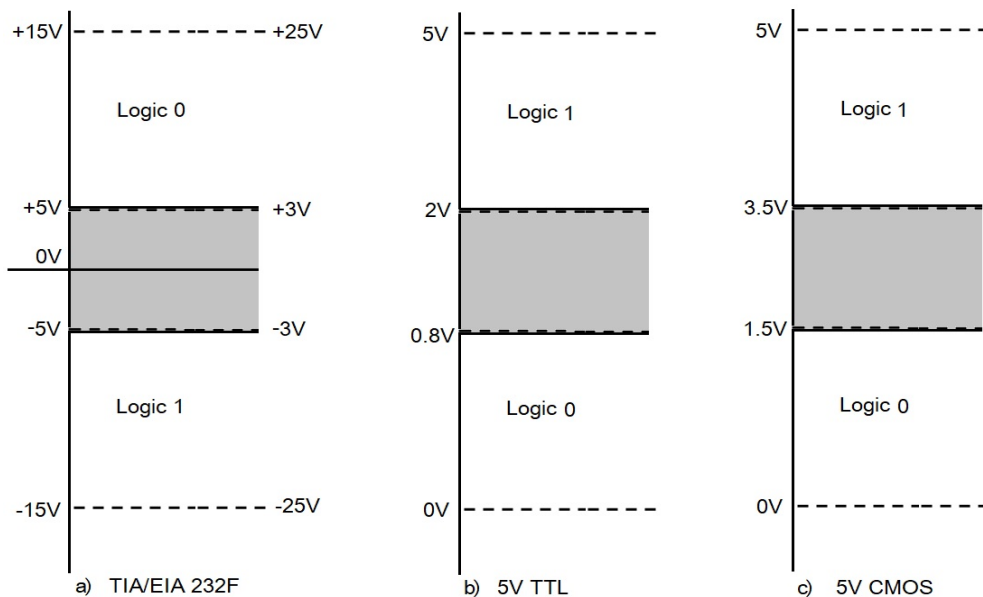


Fig 6.14 Voltage levels for RS232, TTL and CMOS devices.

6.5 Voltage level converters for RS 232 to TTL, CMOS

Microprocessor and microcontrollers are typically TTL or CMOS logic devices. The logic levels of RS 232 standard, TTL and CMOS vary from one another. The receiver voltage

levels of RS 232 standard have been discussed earlier, which uses -3 to -25 for logic 1 and +3 to +25 for logic 0. Typical TTL or CMOS devices are 5V devices which have varied voltage levels for logic 1 and logic 0. In addition to this, various other TTL and CMOS devices exist that operate on different voltages, like 3.3V, 2.4V, 1.8V, 1.2V etc. The PIC18 devices could be operated in different voltage range i.e. 2V to 5.5V.

Due to the variation in voltage levels across RS 232, TTL and CMOS devices, appropriate voltage level converters are required for using RS 232 standards with devices that operate on different voltage levels. These converters are also called as transceivers. A summary of voltage levels of RS232, TTL and CMOS devices is presented here in fig 6.14.

Various transceivers are available from different manufacturers. A list of different transceivers is shown in table 6.4.

Table 6.4: Summary of Transceivers

S. No	Device	TX	RX	Ext Cap	Standard	Data Rate Kbps	Operating Voltage	Auto Shutdown
1	TI - MAX232, MAX232x	2	2	Y	TIE/EIE-232-F	120	5V	N
2	MI - MAX225, MAX233, MAX235, MAX245, MAX246, MAX247	*	*	N	TIE/EIE-232-E	120	5V	*
3	MI - Rest of MAX220 to MAX249	*	*	Y	TIE/EIE-232-E	120	5V	*
4	TI - TRS3318E	2	2	Y	Upgraded RS232	460	2.25 to 3V	Y
5	TI - TRS3122E	2	2	Y	Upgraded RS232	1000	1.8, 3.3, 5V	Y

TX – Drivers, RX – Receivers, Ext Cap – External Capacitor required, Y –Yes, N – No and * indicates variation among devices.

a) Texas Instruments MAX 232x

The TI's MAX232x is a dual EIA-232 transceiver (two drivers and two receivers). It meets and exceeds the TIA/EIA-232-F standard, is operable on single power supply, VCC of 5V with 1 μ F charge pump capacitors with $\pm$ 30V input levels which exceeds the RS232F standard of $\pm$ 25V. It supports data rate up to 120,000 bits/s and provides ESD protection. The transceiver and receiver provide inverted signals i.e a high level on TIN produces a Low on TOUT and a Low on RIN produces a High on ROUT.

b) Maxim Integrated MAX 220 to MAX249

Maxim Integrated provides different variations of transceivers such as MAX220 to MAX249. The MAX220 to MAX249 series contain wide range of transceivers which vary based on the external components used, number of drivers, receivers, enable signals etc. A summary of features of Maxim's series is presented in Table 6.8. These transceivers meet the TIA/EIA-232-E standard, are operable on single power supply, VCC of 5V with 1 μ F charge

pump capacitors. Most of the devices of MAX220 to MAX249 range support 120,000 bps. Similar to TI devices MI's transceiver and receiver provides inverted signals i.e. a high level on TIN produces a Low on TOUT and a Low on RIN produces a High on ROUT.

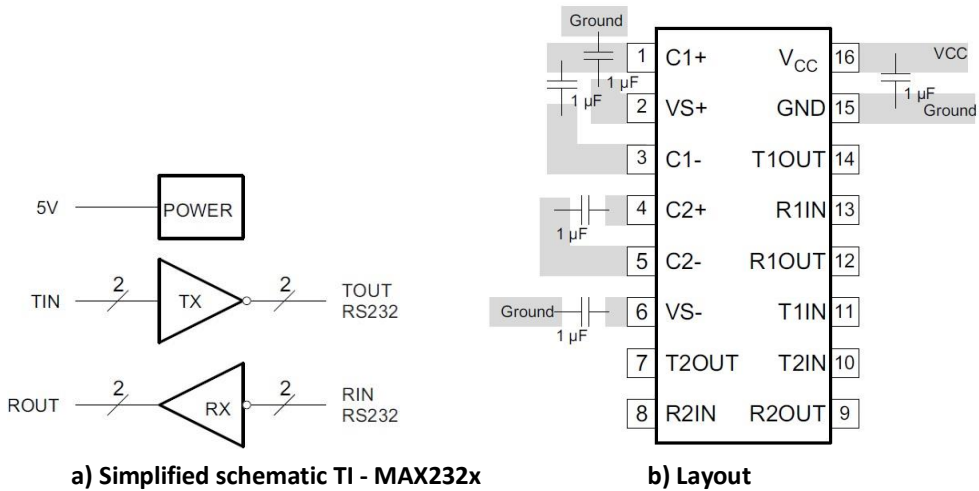


Fig 6.15 Simplified schematic & layout of MAX232x

Translates between TIA/EIE-232-F & TTL/CMOS levels

(Source: TI - SLLS047M: MAX232x Dual EIA-232 Drivers/Receivers)

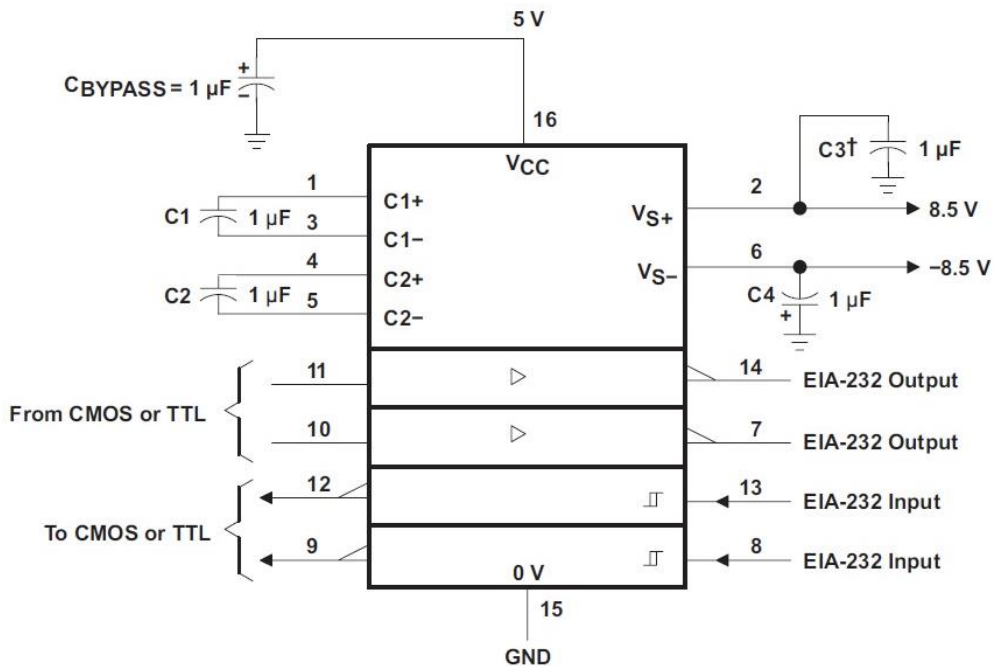


Fig 6.16 Typical operating circuit of TI - MAX232x

(Source: TI - SLLS047M: MAX232x Dual EIA-232 Drivers/Receivers)

Table 6.5: Absolute Maximum Ratings

Parameter		Min	Max	Unit
VCC Input Supply voltage range		-0.3	6	V
VS+ Positive output supply voltage range		VCC - 0.3	15	V
VS- Negative output supply voltage range		-0.3	-15	V
VI Input voltage range	T1IN, T2IN	-0.3	VCC + 0.3	V
	R1IN, R2IN		±30	
VO Output voltage range	T1OUT, T2OUT	VS- - 0.3	VS+ + 0.3	V
	R1OUT, R2OUT	-0.3	VCC + 0.3	

Table 6.6: Recommended Operating Conditions

Parameter	Min	Typical	Max	Unit
VCC Supply voltage	4.5	5	5.5	V
VIH High-level input voltage (T1IN,T2IN)	2			V
VIL Low-level input voltage (T1IN, T2IN)			0.8	V
R1IN, R2IN Receiver input voltage			±30	V

Table 6.7: TI's MAX232x Pin functions

PIN		Type	Description
Name	No		
C1+	1	—	Positive lead of C1 capacitor
VS+	2	O	Positive charge pump output for storage capacitor only
C1-	3	—	Negative lead of C1 capacitor
C2+	4	—	Positive lead of C2 capacitor
C2-	5	—	Negative lead of C2 capacitor
VS-	6	O	Negative charge pump output for storage capacitor only
T2OUT, T1OUT	7, 14	O	RS232 line data output (to remote RS232 system)
R2IN, R1IN	8, 13	I	RS232 line data input (from remote RS232 system)
R2OUT, R1OUT	9, 12	O	Logic data output (to UART)
T2IN, T1IN	10, 11	I	Logic data input (from UART)
GND	15	—	Ground
VCC	16	—	Supply Voltage, Connect to external 5V power supply

For data transfers without the need for handshaking signals, transceivers like MAX220, MAX222, MAX231, MAX232, MAX232A, MAX233, MAX233A, MAX242 and MAX243 are preferred as they contain only 2 drivers and receivers. The TI's transceivers presented, are also used in similar case where handshaking signals are not required.

For serial transfers where handshaking signals are required, transceivers such as MAX223, MAX 225, MAX235, MAX236, MAX237, MAX238, MAX239, MAX 240, MAX241 and MAX244 to MAX249 are a better option as they provide over 4 transmitters/receivers.

Table 6.8: TI's MAX232x Pin functions

PART NUMBER	POWER SUPPLY (V)	NO. OF RS-232 TX / RX	NO. OF EXT CAP	SHDN & THREE-STATE	Rx ACTIVE IN SHDN	DATA RATE (kbps)	FEATURES
MAX220	+5	2/2	4	No	No	120	Ultra-low-power, industry-standard pinout
MAX222	+5	2/2	4	Yes	No	200	Low-power shutdown
MAX223	+5	4/5	4	Yes	Yes	120	MAX241 and receivers active in shutdown
MAX225	+5	5/5	0	Yes	Yes	120	Available in SO
MAX230	+5	5/0	4	Yes	No	120	5 drivers with shutdown
MAX231	+5, +7.5	2/2	2	No	No	120	Standard +5/+12V or battery supplies; same functions as MAX232
MAX232	+5	2/2	4	No	No	120 (64)	Industry standard
MAX232A	+5	2/2	4	No	No	200	Higher slew rate, small caps
MAX233	+5	2/2	0	No	No	120	No external caps
MAX233A	+5	2/2	0	No	No	200	No external caps, high slew rate
MAX234	+5	4/0	4	No	No	120	Replaces 1488
MAX235	+5	5/5	0	Yes	No	120	No external caps
MAX236	+5	4/3	4	Yes	No	120	Shutdown, three state
MAX237	+5	5/3	4	No	No	120	Complements IBM PC serial port
MAX238	+5	4/4	4	No	No	120	Replaces 1488 and 1489
MAX239	+5, +7.5	3/5	2	No	No	120	Standard +5/+12V or battery supplies; single-package solution for IBM PC serial
MAX240	+5	5/5	4	Yes	No	120	DIP or flatpack package
MAX241	+5	4/5	4	Yes	No	120	Complete IBM PC serial port
MAX242	+5	2/2	4	Yes	Yes	200	Separate shutdown and enable
MAX243	+5	2/2	4	No	No	200	Open-line detection simplifies cabling
MAX244	+5	8/10	4	No	No	120	High slew rate
MAX245	+5	8/10	0	Yes	Yes	120	High slew rate, int. caps, two shutdown
MAX246	+5	8/10	0	Yes	Yes	120	High slew rate, int. caps, three shutdown
MAX247	+5	8/9	0	Yes	Yes	120	High slew rate, int. caps, nine operating
MAX248	+5	8/8	4	Yes	Yes	120	High slew rate, selective half-chip enables
MAX249	+5	6/10	4	Yes	Yes	120	Available in quad flatpack package

(Source: Maxim Integrated - 19-4323; Rev 18; 5/19, MAX220-MAX249)
(+5V-Powered, Multichannel RS-232 Drivers/Receivers)

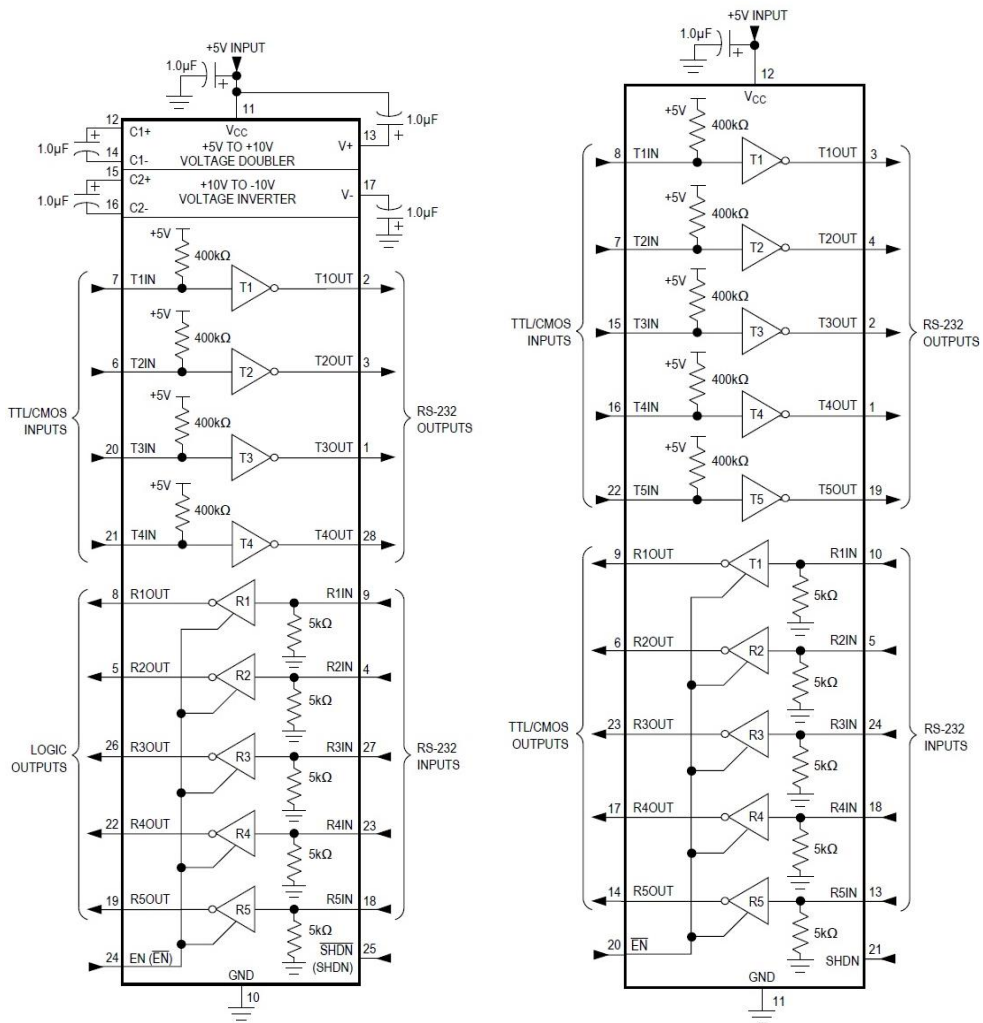


Fig 6.17 Typical operating circuit of Maxim Integrated's: a) MAX223, b) MAX235

(Source: Maxim Integrated - 19-4323; Rev 18; 5/19, MAX220-MAX249)

(+5V-Powered, Multichannel RS-232 Drivers/Receivers)

When minimal PCB footprint size is a priority, transceivers like MAX225, MAX233, Max235, MAX245, MAX246 or MAX247 are better suited since they do not require external capacitors for charge pump etc. The fig 6.17a shows typical operating circuit of MAX223 which uses external capacitor components and the fig 6.17b shows MAX235 which does not use any external components as all the required ones are built in the IC. Some transceivers additionally provide control enable signal for reception or transmission or both. The fig 6.17 shows such feature where only receivers are provided with control enable signal.

Additionally, they may also have shut down features as in MAX222, MAX223, MAX225, MAX236, MAX242 and MAX245.

Table 6.9: Absolute Maximum Ratings of TRS3122E

Parameter		Min	Max	Unit
V _{CC} , Charge pump power supply		−0.3	6	V
V _L , Logic power supply		−0.3	6	V
V ₊ , Positive storage capacitor voltage		−0.3	7	V
V _− , Negative storage capacitor voltage		0.3	−7	V
(V ₊) + (V _−) Absolute Difference			13	V
V _I , Input voltage	FORCEOFF, FORCEON	−0.3	6	V
	DIN	−0.3	V _L + 0.3	
	RIN (0Ω series resistance)		±20	
	RIN (≥250Ω series resistance)		±25	
V _O , Output voltage	DOUT		±13.2	V
	ROUT	−0.3	V _L + 0.3	

Table 6.10: Recommended operating conditions of TRS3122E

Parameter			Min	Typ	Max	Unit
VCC Charge pump power supply	Tripler Mode		1.65	1.8	2	V
	Doubler Mode	3	3.3	3.6		
		4.5	5	5.5		
VL Logic power supply			1.65		VCC	V
RIN RS-232 Receiver interface			-15		15	V
DOUT RS-232 Transmitter interface			-12		12	V
VIL GPIO Input logic threshold low	DIN, FORCEOFF, FORCEON	VL = 5.0V	0		1.7	V
		VL = 3.3V	0		1.1	
		VL = 1.8V	0		0.6	
VIH GPIO Input logic threshold high	DIN, FORCEOFF, FORCEON	VL = 5.0V	3.3		VL	V
		VL = 3.3V	2.2		VL	
		VL = 1.8V	1.2		VL	
VOZ ROUT disabled	FORCEOFF = 0V		0		VL	V

c) Texas Instruments - TRS3122E: Low Power 1.8 V Dual RS-232 Transceiver

The TI's TRS3122E is a low power dual RS232 transceiver (two drivers and two receivers). It is operable with VCC of 1.8V, 3.3V or 5V and eliminates need for different transceivers when PIC18 is operated with different voltages.

The absolute maximum ratings & recommended operating conditions are presented in table 6.9 and 6.10. The typical operating circuit of TI - TRS3122E is shown in fig 6.18. Similar to above discussed transceivers; it is equipped with charged pump - unique

Tripler charge pump (3 flying capacitors instead of 2) architecture, ESD protection. It supports data rate up to 1000 k bits/s and an Auto Power down plus feature.

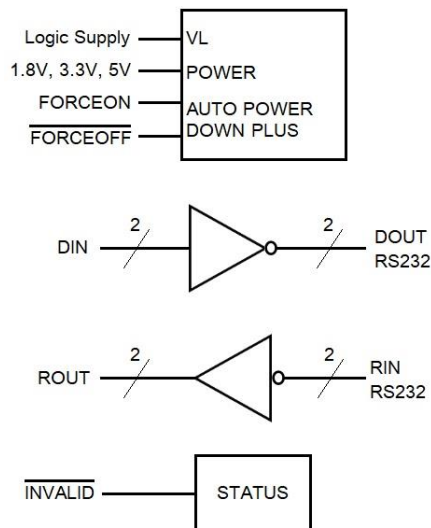


Fig 6.18 Typical operating circuit of TI - TRS3122E: Low Power Dual RS-232 Transceiver

(Source: TI - SLLSET7C: TRS3122E Low Power Dual RS-232 Transceiver)

Table 6.11: Auto Power down plus Function modes

Driver DIN	FORCEON	<u>FORCEOFF</u>	Elapsed Time	Output DOUT	Driver Status
x	x	L	x	Z	Powered OFF
L	H	H	x	H	Normal operation, Auto Power down plus disabled
H	H	H	x	L	
L	L	H	<30 s	H	Normal operation, Auto Power down plus enabled
H	L	H	<30 s	L	
L	L	H	>30 s	Z	Powered OFF by Auto Power down plus feature
H	L	H	>30 s	Z	
Receiver RIN		<u>FORCEOFF</u>	Elapsed Time	Output DOUT	Receiver Status
x		L	x	Z	Powered Off
L		H	x	H	Normal operation, Auto Power down plus disabled/enabled
H		H	x	L	
Open		H	x	H	

L = Low Level, H = High Level, Z = High Impedance (OFF), x = H or L value

Note: INVALID Status is low if the RIN1 and RIN2 is open, else it is high.

The Auto Power down plus is an attractive feature that activates low power mode (0.5µA consumption) when device is inactive for more than 30 seconds with a wakeup time

between 30 μ s and 150 μ s. It is ideal for reduced power consumption in power sensitive applications or battery powered systems. The Auto Power down plus Function modes is presented in table 6.11.

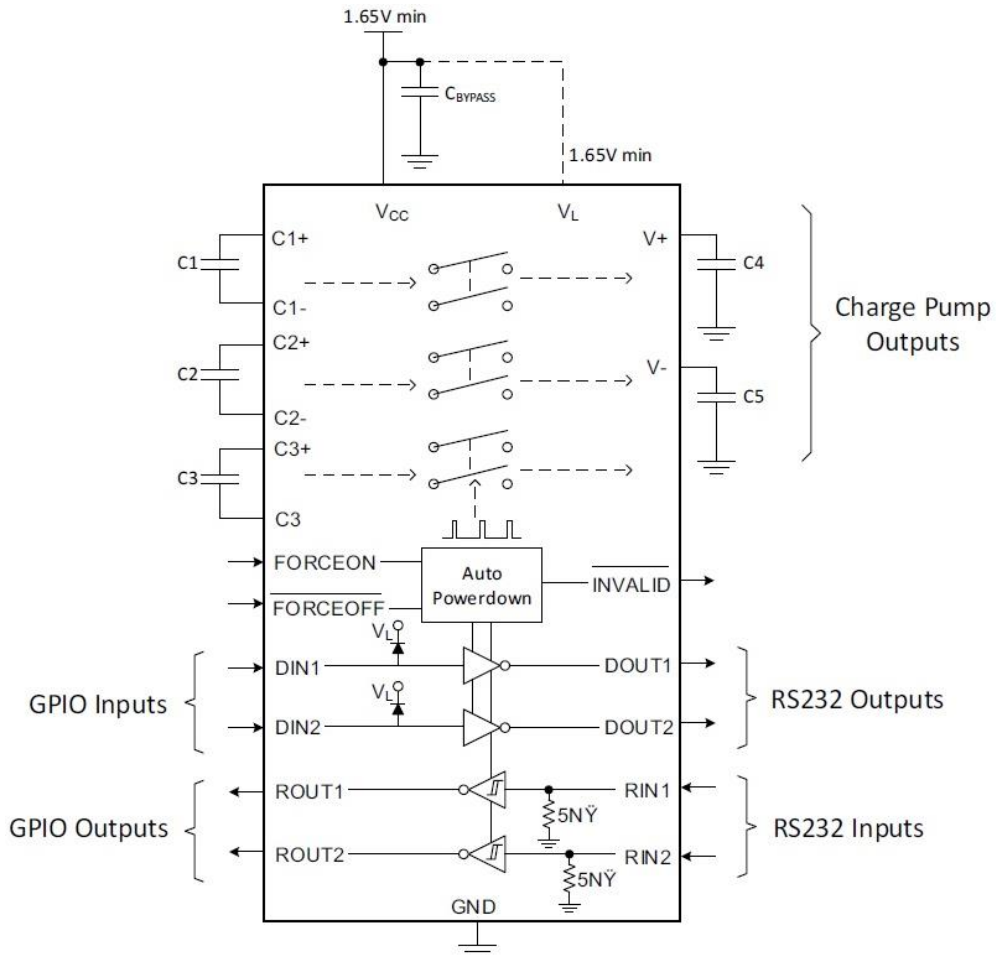


Fig 6.19 Typical schematic of TI - TRS3122E

(Source: TI - SLLSET7C: TRS3122E - Low Power Dual RS-232 Transceiver 1.8 V)

A typical schematic of TRS3318E is presented in fig 6.19. The transceiver and receiver provide inverted signals i.e. a high level on TIN produces a Low on TOUT and a Low on RIN produces a High on ROUT. For a 2.5V single supply application, the Ti's TRS3318E is recommended.

For distances up to 50ft, systems use RS 232. However, for shorter distances serial communication using RS 232 has mostly been replaced by USB. Due to wide use of USB, RS 232 to USB converters, USB to CMOS/TTL level converters are also commonly required along

with RS232 to CMOS/TTL levels. The details of voltage levels of USB are discussed separately in USB section of Ch. 15.

6.6 Interfacing signals

A microprocessor or a microcontroller-based system is formed by interfacing various devices (sensors, actuators, peripherals, other MP or MC) and memories. The parallel data transfer and serial data transfer discussed in the earlier sections of this chapter demonstrate the concepts of the same. In addition to the handshaking signals discussed in the respective sections, the MP or MC based systems generate different other control signals that are used in the interfacing of different devices. This section introduces the various signals that are typically used for interfacing. The pins of MP/MC's typically contain various types of signals. They are grouped as Address lines, Data lines, Control lines, Status lines, Power lines and peripheral or special functionality. To reduce the pin count of an IC, multiple functionalities are multiplexed into each pin.

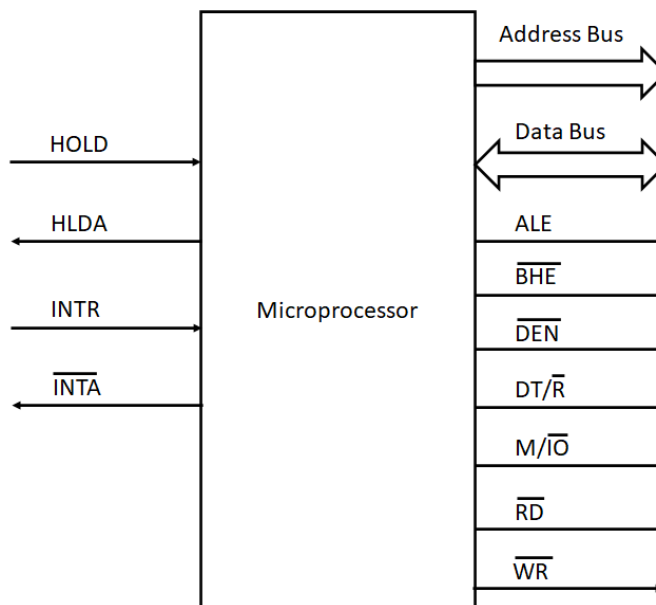


Fig 6.20 Typical Microprocessor Interfacing signals

This section presents some of the most used interfacing signals. A list of some commonly used interfacing signals of classic 8086 microprocessor is presented in table 6.12 and block diagram representation is shown in fig 6.20. Similar functionalities are available in PIC18 microcontrollers that have external memory interface. The actual port details, signals of PIC18 are presented in Ch8 and details of external memory interface in Ch.8.

Table 6.12: Interfacing pins of an 8086 microprocessor

Signal Name	Description
AD0-AD15	Address/Data Lines
A0-A15	Address Lines
D0-D15	Data Lines
ALE	Address Latch Enable
<u>DEN</u>	Data Enable
<u>DT/R</u>	Data Transmit or Data Receive
<u>M/IO</u>	Memory or IO - Input Output
<u>RD</u>	Read
<u>WR</u>	Write
BHE	Bus High Enable
A0	LSB Bit of Address Bus
A1	LSB+1 Bit of Address Bus
RDY	Ready
INTR	Interrupt Request
<u>INTA</u>	Interrupt Acknowledge
HOLD	Hold
HLDA	Hold Acknowledge

AD0:AD15 – Address/Data lines, A0:A15 – Address lines, D0:D15 – Data lines

A system bus is a combination of address, data, control and status lines that is external to the IC and to which all the devices are generally connected. Each device or a location of memory in an MP/MC based system is identified by an address. This address is typically generated by the MP/MC and is transferred on the address bus. Similarly, information or data may be generated and transferred but is bi-directional as data can be sent by a MP/MC or any device or memory in the system. This information or data is transferred on data bus. These parallel address bus lines are represented as A0:A15 and data bus lines are represented as D0:D15.

To reduce the number of pins and parallel lines, address and data are normally multiplexed together on a set of common parallel lines. It is termed as Address/Data lines and is represented as AD0:AD15. The numbering AD0:AD15 indicates 16 lines numbered from AD0, AD1, AD2 up to AD15 in a 16-bit bus system. The 8085 consists of AD0:AD15 whereas 8086 has AD0-AD15 and A16 to A19, and PIC18's that have external memory interface uses AD0:AD19.

A1, A0: Lower two bits of Address bus

As discussed earlier, AD0:AD15 are multiplexed address/data bus where A0 to A15 are the address bits. The A1, A0 are the two lower order bits of address bus. The devices that can perform operations on more than 8-bits typically (legacy 8086 16-bit Microprocessor)

divide the memory into different banks, i.e. odd bank and even bank. All the memory locations in an odd bank have an odd address and even bank consists of all even addresses. A similar interfacing is shown in fig 6.21.

The odd or even memory bank is identifiable through the A0 bit. Even memory bank is selected when the value of the address bit A0 (LSB) is 0 and odd bank is selected when it is A0 is 1. The 8086 allows for accessing the odd or even address of program or data memory, peripherals and I/Os.

The Program counter of PIC18 which has 16-bit program memory neglects the A0 bit for accessing the 16-bit instruction word and accesses the 16-bit word always starting from an even address. However, it allows for accessing the odd or even address of 8-bit data memory, EEPROM, peripherals and I/Os.

Processors like ARM which operate on 32-Bit or more divide memory into 4 banks, bank 0, bank 1, bank 2 and bank 3. The four banks are identifiable through the two lower order bits of the address A1 and A0. The A1, A0 combination of 00 identifies bank 0, 01 for bank 1, 10 for bank 2 and 11 for bank 3.

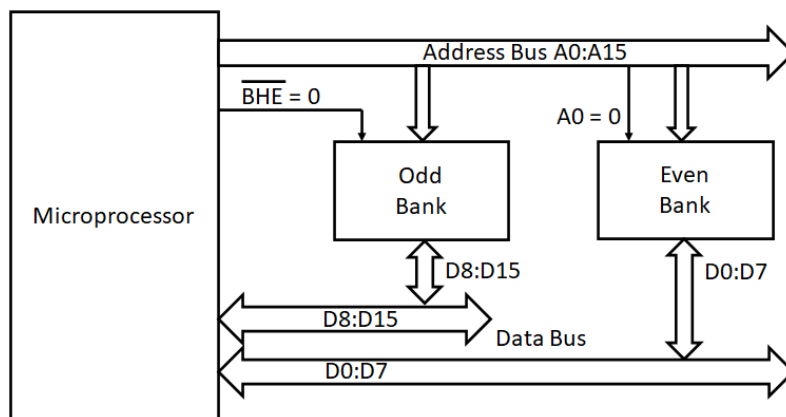


Fig 6.21 Simplified structure of Even/Odd memory bank interface

BHE: Bus High Enable

The BHE – Bus High Enable signal is generated by the microprocessor to indicate the transfer of data on the higher 8-bit of data bus where D0:D7 form the low 8-bit data bus and D8:D15 form the high 8-bit data bus. The low 8-bit data bus of 8086 is connected to the even memory bank and high 8-bit data bus of 8086 is connected to odd bank where access of data from odd address always uses the high 8-bit data bus D8:D15. The BHE signal is made low by the microprocessor when an 8-bit value is accessed from an odd address or when a 16-bit value is accessed which accesses both odd and even address banks.

ALE: Address Latch Enable

As discussed earlier, the Address/Data lines are time multiplexed together to reduce the number of required I/O lines in a MP or MC. Typically a machine cycle consists of 4 to 5 clock cycles denoted as T1, T2, T3, Tw and T4. The AD0:AD15 lines are used to transfer address during the first clock cycle and later for transferring of data during T3 and T4. An active high signal is asserted on ALE pin during T1 cycle which indicates the presence of address on the AD0:AD15 lines. Since the address is available only during T1, the address must be stored into the 74LS373 latch to make it available during T2, T3, Tw and T4 cycles for device identification and activation. The ALE pin acts as an enable signal for storing the address that is available on AD0:AD15 lines during T1 cycle on to the latch. The interfacing of ALE is demonstrated in the interfacing shown in fig 6.22.

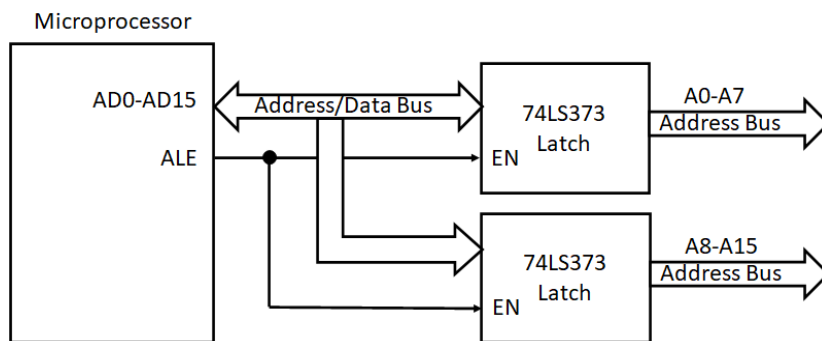


Fig 6.22 Latching of address into 74LS373 latch using ALE signal

DEN: Data Enable

The DEN – Data Enable signal is opposite of the ALE signal. While ALE is used to indicate the presence of address, the DEN is instead used to indicate the presence of data on the AD0:AD15 lines. An active low signal is asserted on DEN pin during T3, Tw and T4 which indicates the presence of data on the AD0:AD15 lines. Microprocessors are typically not designed for driving loads beyond few tens of milli-amps. The bi-directional buffers 74x245 are hence connected to the data bus to ensure the drivability and to maintain the required logic levels of the data bus. The DEN signal is used to enable these bi-directional buffers.

DT/R: Data Transmit/Receive

The DT/R represents data transmit or receive. The bi-directional buffers discussed earlier can transmit the data in both the directions (Left to Right or Right to Left). A pin is provided with these buffers for setting the direction. The value applied at this pin typically determines the direction of data transfer. The interfacing of DT/R and DEN is demonstrated in the interfacing shown in fig 6.23.

While DEN enables the bi-directional buffers, the direction of the buffer is set using the DT/R signal. When set to logic 0 the data is read and received (right to left) by the microprocessor whereas as when it is set to 1 the data is transmitted (left to right) by microprocessor and the connected devices are driven by the buffer.

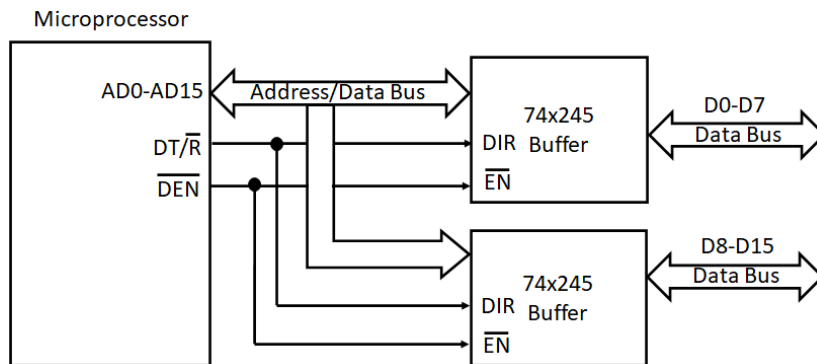


Fig 6.23 Driving data through 74x245 bi-directional buffer using DT/R and DEN signals

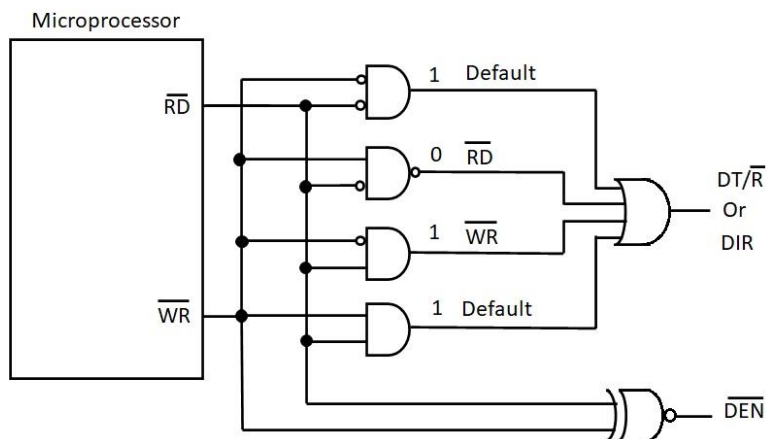


Fig 6.25 Data Transmit/Receive or Direction, Data Enable signal generation

RD: Read

A read signal RD is generated by a microprocessor when an I/O or memory must be read. It is typically an active low signal and an active low signal on this pin enables the memory, I/O or peripheral to place the data on the data bus. The data placed on the data bus is then read by the microprocessor.

WR: Write

A write signal WR is generated by a microprocessor when data must be written to an I/O or memory. It is typically an active low signal. An active low signal indicates transfer of data on to the data bus by the microprocessor. This signal is used by the memory, I/O

devices or peripherals to store the data that is present on the data bus. The interfacing of RD and WR signals to generate DT/R or DIR and DEN signals is demonstrated in the interfacing shown in fig 6.24.

M/IO: Memory or Input/Output

The M/IO is a control signal generated by a microprocessor. It is used to differentiate between memory and Input/Output. It is used along with read, write signals to generate Memory read, Memory write, I/O read, I/O write signals. The interfacing of RD, WR and M/IO signals to generate memory read MRD, memory write MWR, IO read IORD, IO write IOWR is demonstrated in the interfacing shown in fig 6.25.

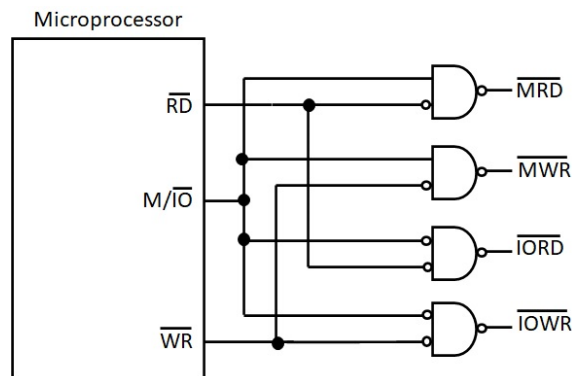


Fig 6.24 Memory Read/Write and IO Read/Write control signal generation

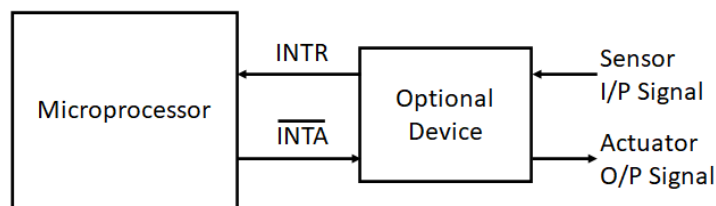


Fig 6.26 Sensor/Actuators interfacing using interrupt

INTR: Interrupt Request

The $\overline{INTR}$ is an interrupt request signal that is generated by an external peripheral. This is a signal of high importance, used to stop the processor or controller from executing its current task and to make it perform the higher important task related to that peripheral.

INTA: Interrupt Acknowledge

When a processor or controller receives an active high signal on $\overline{INTR}$ pin, it finishes the execution of current instruction and generates an active low signal on the $\overline{INTA}$ pin as an acknowledgement to the $\overline{INTR}$. This indicates to the peripheral that the interrupt is detected, and the corresponding task will be executed. Additionally, when external interrupt controller

like 8259 is used, this signal acts as strobe to make the 8259 place the interrupt vector on to the data bus so that execution is transferred to the respective interrupt vector's location. The details of 8259 are discussed later in traditional peripherals section of this chapter. The interfacing of INTR and INTA is demonstrated in the interfacing shown in fig 6.26.

HOLD: Hold

Hold is a pin function of microprocessor to which a device such as a DMA – Direct Memory Access is connected. The DMA generates an active high signal to the HOLD input pin of MP to request for access of system bus when the DMA itself had earlier received a request from any of the I/O's.

HLDA: Hold Acknowledge

When an active high signal is received by the microprocessor on the HOLD pin, the processor generates a hold acknowledge signal by sending an active high signal on the HLDA pin as a response to the HOLD signal. This signal is an indication to the DMA that the microprocessor has released or relinquished the control of the system bus and that the DMA can become the master of the system bus. The interfacing of HOLD and HLDA is demonstrated in fig 6.44 in the traditional peripherals section of 6.8.4

6.7 Memory interfacing concepts

The Microprocessors and microcontrollers are manufactured by different manufacturers and their architecture and organization vary from one another. They are designed to efficiently and effectively utilize limited number of resources like number of registers, ALU, bus structures, features etc. Different manufacturers employ different techniques to achieve the optimum performance for their set of criterions.

This section discusses some of the concepts related to memory which affects the architecture and organization of a processor or a controller. They are Memory Segmentation, banked memory/register, memory interleaving, memory mapped I/O and I/O mapped I/O.

6.7.1 Memory Segmentation

Segmentation is the process of dividing the memory into different logical sections or segments. Microprocessors such as x8086 use the concept of memory segmentation. It has an address generating mechanism which generates the required 20-bit address by a combination of two registers - 16-bit segment and 16-bit offset registers. The 16-bit segment register forms the base address, and the second register contains the offset from the segment base. Hence it is called as offset register and allows for identifying the specific memory location in the segment. This allows for 16 different segments with each segment

of size 65536 bytes. The identification of a location within the segment requires a smaller number of bits and results in other benefits such as reduced number of required bus lines, and size etc. The benefits of memory segmentation are presented here considering the 8086 microprocessors.

- Instructions like INC, DEC are typically used for changing the address value. Such operation requires register size equal to that of the number of address bits. Due to segmentation, the required register size for performing the address related operations is reduced to 16-bit instead of the actual 20-bit.
- Since the registers only contain 16-bit value, the operations such as address increment & decrement requires an ALU of just 16-Bits.
- As the ALU is of 16-Bit size, the internal bus structure required for transferring the address/data is of 16-bit, similar to the registers.
- This allows for reduced number of parallel bus lines, which in-turn reduces the number of bus structure components (like multiplexers) that are required for register bus transfer system.
- Additionally, the 16-bit operations required allows for faster response, consume less power, space as it eliminates the need of additional gates, bus lines and other components that are required for the extra 4-bits in the 20bit operations. However, for performing 20-bit operations, the ALU, registers and bus structures are required to be 20-bit.
- Since the effective physical address is formed by combination of segment and offset register, any change (incrementing or decrementing) of offset value cannot create an effective address that falls into a different segment of memory. A different memory segment can be accessed only by a change of segment value in non-overlapped segments. Even an infinite increment and decrements of offset register do not create access into a different segment unless the segment value is changed. Hence the memory is protected from any overrun and allows for isolation of segments. A representation of non-overlapping segments is presented in fig6.27a.
 - This isolation or protection of memory is implemented in a different way in later versions of PIC18 and latest processors or controllers where sections of memory are protected from accessing without permissions etc.
- It also allows for flexibility of numerous overlapping segments in cases where two certain areas of memory are required across different segments. As an example, an application program may have a set of constants or function codes that may be required in different segments. In such a case the constants or the function codes are made to overlap among segments so that they are accessible while being in

different segments. A representation of overlapping segments is presented in fig6.27b.

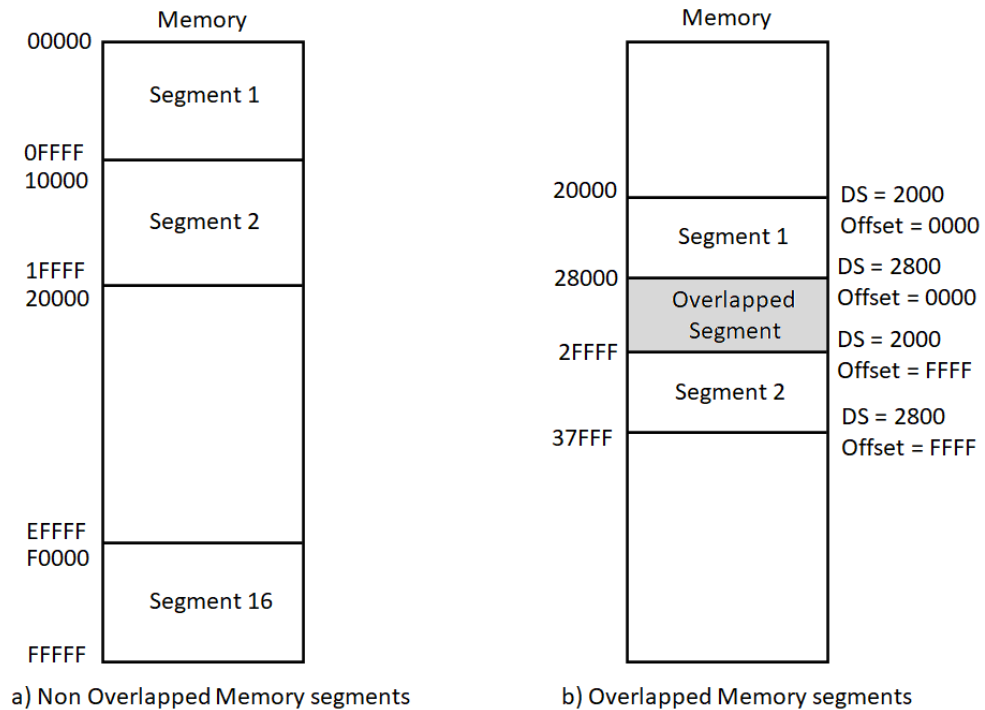


Fig 6.27 Representation of non-overlapped and overlapped memory segments

As 16-bit segment and offset registers are present, the advantage is that a generation of 20-bit address does not require a 20-bit ALU as it is carried out separately by address generation unit. However, the disadvantage is that every time an effective 20-bit address is required, it is only generated after combination of the segment and offset register by a separate address generation unit which is similar to an overhead.

6.7.2 Memory/Register Banks

This concept is similar to that of segmentation but varies based on how the memory is accessed. Many devices employ this concept. The classic and popular 8051 microcontroller employs this concept for RAM. The ARM processors employed by various manufacturers as cores in their processors or controllers employ a banked register concept that is similar to banked memory or banked register concept of 8051. The PIC16, PIC18, and various other devices also employ similar banked concept.

The purpose of banked concept is to reduce the number of bits required to access a memory location or a register. As an example, consider a building with 4 floors and 8 rooms per floor which gives a total of 32 rooms. A unique identification of floor requires 2 bits and

a room in each floor requires 3 bits bringing the total to 5 bits that are required to access a room in the whole building.

Memory Location	Memory	Memory Location	Memory
00000		10000	
00001		10001	
00010		10010	
00011		10011	
00100		10100	
00101		10101	
00110		10110	
00111		10111	
01000		11000	
01001		11001	
01010		11010	
01011		11011	
01100		11100	
01101		11101	
01110		11110	
01111		11111	

Fig 6.28 Representation of non-banked register or memory

Consider a case of identifying a room from outside the building; it requires you to specify the floor number and the room number in that floor which requires all the 5 bits, i.e 2-bits for the floor and 3 bits for the specific room in that floor. Now consider the case where you are already in a floor and need to access any one room in that floor. Now you only require 3 bits to access a specific room as you no longer require the floor details. So, two bits are saved each time you refer to a room in that floor.

Now consider the memory or set of registers being arranged as different floors and each room representing a memory location or a specific register. The concept of accessing a specific memory location or register is similar to the buildings floor and room concept presented earlier. A non-banking concept is shown in fig 6.28 and an equivalent banked register/memory concept is shown in fig 6.29.

Here a bank is equivalent to the floor, and it is selected in advance using one separate instruction. It can then be followed by instructions that utilize reduced number of bits required to access register or memory location in that bank. This reduces the number of bits that must be specified as part of the instruction to access a specific memory location or register. Hence overall the required number of bits for instructions and the program memory size is generally reduced along with the required bit operations.

Bank/Floor	Room/Register/ Memory Location	Memory	Bank/Floor	Room/Register/ Memory Location	Memory
00	000		10	000	
	001			001	
	010			010	
	011			011	
	100			100	
	101			101	
	110			110	
	111			111	
01	000		11	000	
	001			001	
	010			010	
	011			011	
	100			100	
	101			101	
	110			110	
	111			111	

Fig 6.29 Representation of banked registers/memory

6.7.3 Memory Interleaving

Similar to the concept of segmentation and banking, memory interleaving is a process of organizing the memory into different block or banks. The purpose of the memory interleaving is to generally ensure hierarchical or systematic division or combination of memory. i.e. it is used to modularize the large amount of memory into blocks or create a large amount of memory by an interconnection of smaller memory blocks. This allows flexibility of scaling up/down of memory in a system. Such a setup requires successive data bytes to be placed in continuous memory locations in the same block of memory as shown in fig 6.30.

This technique is also used to ensure that higher number of bytes is transferred to devices that operate on data size of 16-bit and above with 8-bit memory blocks. The purpose of this is to have multiple memory blocks transfer data in parallel. This is achieved when successive data bytes are available in successive memory blocks that can place the data on the data bus in parallel as shown in fig 6.31. This technique also applies for a device that operate on data size of 32-bit and above with 8 or 16-bit memory blocks.

Various book authors of 8086 uses the terminology of odd bank and even bank for interleaved memory blocks. This is different to the actual banking concept discussed earlier. In 8086 the memory is organized as odd and even memory banks where the odd bank consists of all the odd memory locations and even bank consists of all the even memory locations. i.e. successive data bytes are available in successive memory blocks. The respective odd or even bank or both is accessed through the A0 and BHE signals. Refer to

the interfacing signals for description of these signal pins. Here the even bank places an 8-bit data on the lower data bus D0-D7 whereas odd bank places and 8-bit data on the higher 8-bit data bus D8-D15.

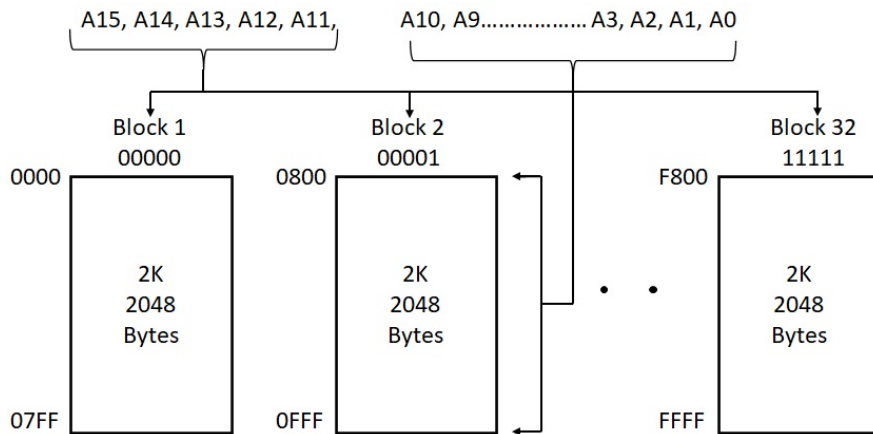


Fig 6.30 Consecutive bytes in a block

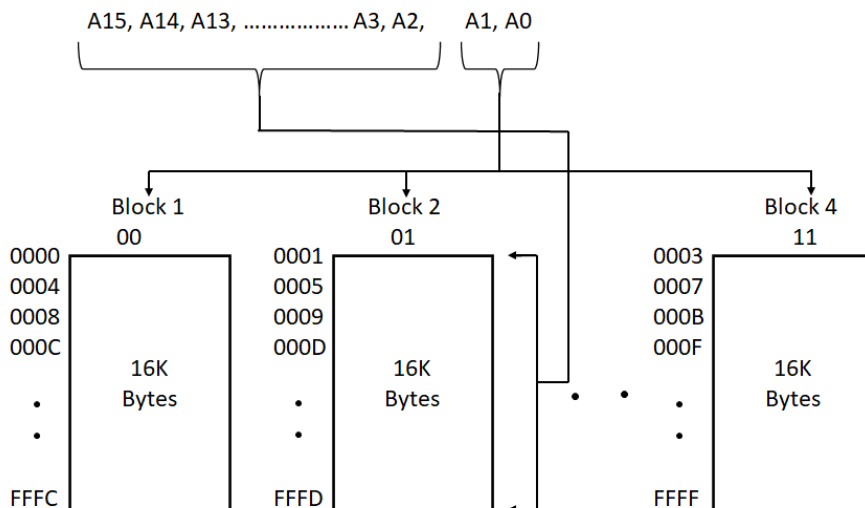


Fig 6.31 Consecutive bytes in consecutive blocks

A similar concept is employed in PIC18's program memory. Processors like ARM which operate on 32-Bit or more divide memory into 4 banks, bank 1, bank 2, bank 3 and bank 4 where the successive blocks contain successive addresses of memory. i.e. block 1 contains locations 0, 4, 8, etc., block 2 contains locations 1, 5, 9 etc., block 3 contains locations 2, 6, 8 etc. and block 4 contains 3, 7, A etc. These four blocks provide 4 bytes in parallel, as example memory location 4 in bank 1, 5 in bank 2, 6 in bank 3 and 7 in bank 4. A typical structure of a 32-Bit device is shown in fig 6.32.

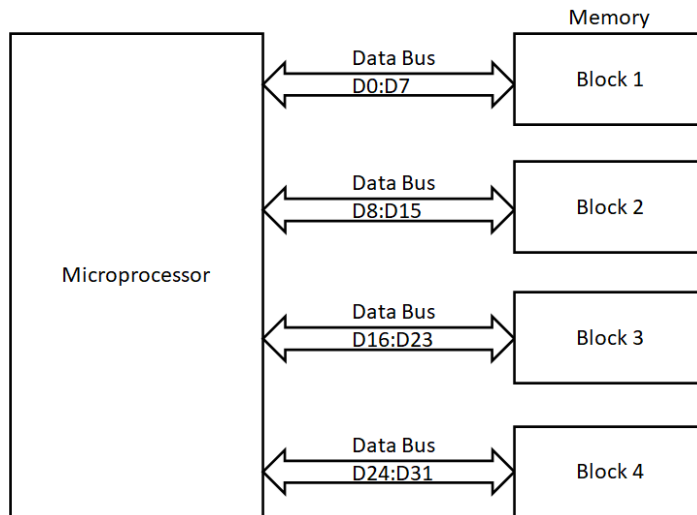


Fig 6.32 A 32-Bit device (ARM) with consecutive bytes in consecutive memory blocks

6.7.4 Mapping Techniques

There are two main mapping techniques.

- 1) Memory mapped I/O &
- 2) I/O mapped I/O

6.7.4.1 Memory Mapped I/O

Memory mapped I/O is the most used mapping technique. This technique treats all the devices, I/O's, peripherals connected to the microprocessor as an individual or block of memory locations and hence no distinction between memory and I/O. Since it treats every connected device also as memory, the $\overline{M}/\overline{IO}$ signal is of little use and could be neglected.

Such a technique utilizes only the memory related instructions and does not use the I/O related instructions like IN/OUT, as I/O's are also considered as memory. For a 16-bit address based device, the addressability (number of devices that can be addressed or uniquely identified) is $2^{16} = 65536$ locations of memory, I/O etc combined. The available 65536 locations are shared among the memory, I/Os etc and no two devices, Memory, I/O etc cannot have the same address. For 8086 it is 20-bit address bus, hence addressability is $2^{20} = 1\text{MB}$.

This above discussion is valid for Von-Neuman architecture where only one bus structure exists. Here the program memory or ROM and data memory or RAM uses distinct addresses in a common address space. Harvard architecture has separate bus structure for program memory and data memory allowing parallel access of instructions from ROM and data from RAM. In such a case the address space of data memory is in addition to the

discussed general address range where data memory and program memory can have same addresses as they have different separate bus and address space. The ports and peripherals that are present in microcontrollers but not in microprocessors share the data memory address space.

6.7.4.2 I/O Mapped I/O

I/O mapped I/O is an alternate mapping technique that is used for increased addressability or to have a clear distinction of addresses between memory and I/O. This technique treats the devices as their own, i.e. memory is considered as it is and similarly I/O, peripherals or other devices are treated as I/O's only. A microprocessor like 8086 has separate set of instructions for accessing I/O's and separate instructions for memory.

Since I/Os are differentiated from memory, the M/IO pin is required to differentiate between the memory and I/O. Memory is accessed using instructions such as "MOV". The memory related instructions make the microprocessor to generate logic 1 on the M/IO pin indicating access of memory. I/O devices are accessed using instructions such as "IN" or "OUT". The I/O related instructions make the microprocessor to generate logic 0 on the M/IO pin indicating access of I/O devices.

For a 16-bit address based device, the memory addressability is $2^{16} = 65536$ locations of memory. A device using such technique additionally uses separate 8-bit or 16-Bit I/O address space. Hence for 8-bit space, the I/O addressability is $2^8 = 256$ I/O's. The 256 I/O's address space is in addition to the 65536 locations of memory. Here the memory and I/O can have same addresses i.e. the first 256 values 00 to FFh. They are differentiated by the memory or I/O instruction that is used which sets or clears the M/IO pin. The 8086 has 20-bit address bus and 16-bit I/O address space. Hence the memory addressability is $2^{20} = 1\text{MB}$ and I/O addressability is $2^{16} = 65536$ I/O's where the memory and I/O can have same addresses i.e. the first 65536 values 0000 to FFFFh.

For Harvard architecture, the address space is a combination of separate program memory address space, data memory address space and I/O memory address space. The ports and peripherals that are present in microcontrollers but not in microprocessors share the data memory address space.

Since the latest processors and controllers that are popularly based on ARM includes address bits of 32 to 64 and since many I/O's are generally not required (most cases <1000) memory mapped I/O is widely used eliminating the need for M/IO signal. Similarly, different instructions and processor architecture exist that eliminate the need for bus high enable BHE signal and instead only use the low address bits like A0, A1 etc for memory interleaving and banking.

6.7.5 Memory Interfacing

Microprocessors typically do not have in-built memory. Microcontrollers have inbuilt memory but are limited by its capacity. Hence interfacing of memory is required. The main memory is typically interfaced through the system bus comprising of parallel lines of address, data and control bus. However secondary memory is generally interfaced through serial communication protocols such as SPI, I2C. Later in chapter 15 different serial protocols are discussed and chapter 8 discusses the PIC18 microcontroller's external memory interface. Here a general parallel line interfacing of memory is presented.

The PIC18 has 21-bit address bus & 16-bit instruction data bus for program memory and 12-bit address bus & 8-bit information data bus for data memory. However, the following discussion here presents generalized examples that consider the general interfacing signals discussed in section 6.6 and assumes that 16-bit address bus & 8-bit or 16-bit data bus is present and presents different mapped I/O's.

Example 1: Interface a memory of 2K bytes as a single block of memory. Consider 16-bit address & 8-bit data bus.

Sol: First list out the possible address values for the 16-bit or the possible combinations as shown below. The possible addresses are 0's at all the address bits to all 1's at all the address bits which makes the address range 0000H to FFFFH.

A 15	A 14	A 13	A 12		A 11	A 10	A 9	A 8		A 7	A 6	A 5	A 4		A 3	A 2	A 1	A 0	Address
0	0	0	0		0	0	0	0		0	0	0	0		0	0	0	0	0000H
1	1	1	1		1	1	1	1		1	1	1	1		1	1	1	1	FFFFH

Now determine the number bits required for a 2K memory or 2K combinations where $1K = 2^{10} = 1024$. i.e 1K requires 10-bits. To get 2K, a multiplication of 2 is required which is achieved by another bit as each additional bit provides a factor of 2. i.e. $2^{11} = 2K$ and hence 11 bits. So, to achieve 2K bytes all the possible 11-bit combinations are required making the address range from 0000H to 07FFH. i.e. A0 to A10 address bit as shown below.

The higher bits A11 to A15 may remain zero or set to any specific fixed combination but is fixed to all 0's in the above discussed example. This fixed combination is used as the chip select to access any location of the 2K memory. The chip select is shown in fig 6.33a. The complete interfacing circuit is shown in fig 6.33b. A decoder may be used for the chip select. As an example, a decoder 5:32 with a combination of 00000 may be used to generate the required chip select for the one shown in fig 6.33a.

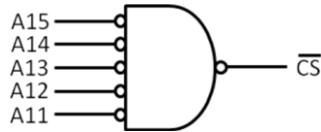


Fig 6.33a Chip Select for 2K Memory

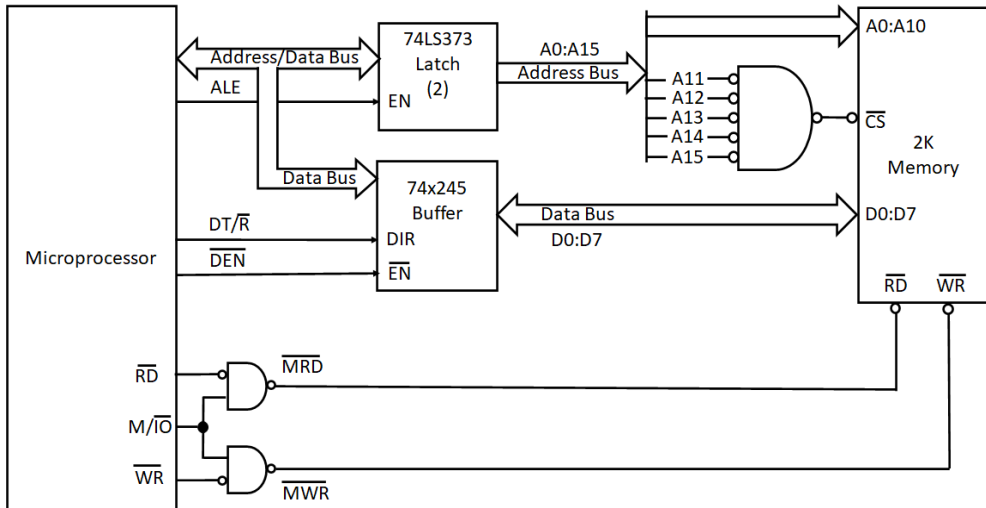


Fig 6.33b Interfacing of 2K memory – Example 1

Note: ROM memory typically is only read and generally does not include a write signal, whereas RAM memory is read and written and hence include read and write signals.

Example 2: Interface a ROM memory of 2K bytes and RAM memory of 4K bytes with each as a single block of memory. Consider 16-bit address & 8-bit data bus.

Sol: We have seen in the example 1 of the possible address values for the 16 address bits which makes the address range 0000H to FFFFH.

2K Byte ROM Allocation → 0000H to 07FFH addresses.

A	A	A	A		A	A	A	A		A	A	A	A		A	A	A	A	Address
15	14	13	12		11	10	9	8		7	6	5	4		3	2	1	0	
0	0	0	0		0	0	0	0		0	0	0	0		0	0	0	0	0000H
0	0	0	0		0	1	1	1		1	1	1	1		1	1	1	1	07FFH
A	A	A	A		A	A	A	A		A	A	A	A		A	A	A	A	Address
15	14	13	12		11	10	9	8		7	6	5	4		3	2	1	0	

0	0	0	0		0	0	0	0		0	0	0	0		0	0	0	0	0000H
0	0	0	0		0	1	1	1		1	1	1	1		1	1	1	1	07FFH

We had also seen that 2K memory requires all possible combinations of 11-bits. Hence addresses in the range of 0000H to 07FFFH are assigned for the 2K memory and assign that as the 2K ROM as shown above. Now that addresses 0000H to 07FFFH are used, the next available address is from 0800H.

A	A	A	A		A	A	A	A		A	A	A	A		A	A	A	A	Address
15	14	13	12		11	10	9	8		7	6	5	4		3	2	1	0	
0	0	0	0		1	0	0	0		0	0	0	0		0	0	0	0	0800H

If the first 2K was not assigned to ROM, the assignment would be $2^{12} = 4K$ bytes of RAM i.e A0 to A11 which indicates required combinations of 12 bits. The addresses from 0800H may be used but increases the complexity of the required chip select signal. As an example, to get 4K from 0800H requires address from 0800H to 17FFFH where 0800 to 08FF have high digit common value of 08H where as 1000H to 17FFFH has high digit common value of 10H. In such a case a chip select may be obtained by logic or of the two common high digit values.

Hence to ensure simplicity we utilize addresses such a way that all the possible lower 12-bit combinations are allocated and hence neglect 0800H to 0FFFFH and start allocation for RAM from 1000H which gives all lower 12 bits combinations. So, addresses 1000H to 1FFFH can be allocated for 4K bytes of RAM which keep the chip select simple to high digit of 1H.

4K Byte RAM Allocation → 1000H to 1FFFH addresses.

A	A	A	A		A	A	A	A		A	A	A	A		A	A	A	A	Address
15	14	13	12		11	10	9	8		7	6	5	4		3	2	1	0	
0	0	0	1		0	0	0	0		0	0	0	0		0	0	0	0	1000H
0	0	0	1		1	1	1	1		1	1	1	1		1	1	1	1	1FFFH

The higher bits A15 to A13 are zero while A12 is 1 for all locations in the selected range of 4K RAM memory. This fixed combination is used as the chip select to access any location of the 2K memory. The complete interfacing circuit is shown in fig 6.34.

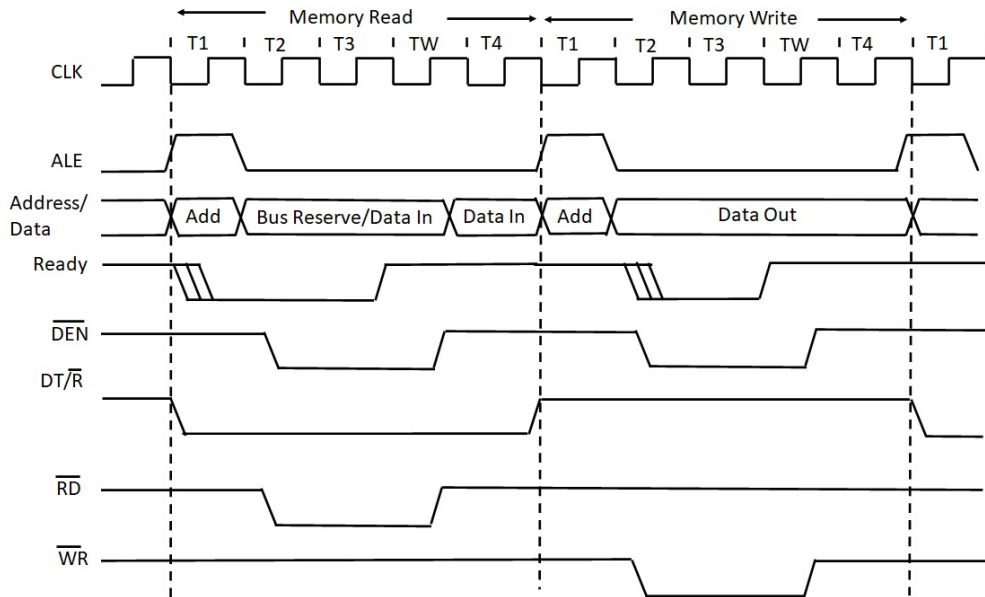


Fig 6.34a Timing diagram of Memory Read and Memory Write - Example 2

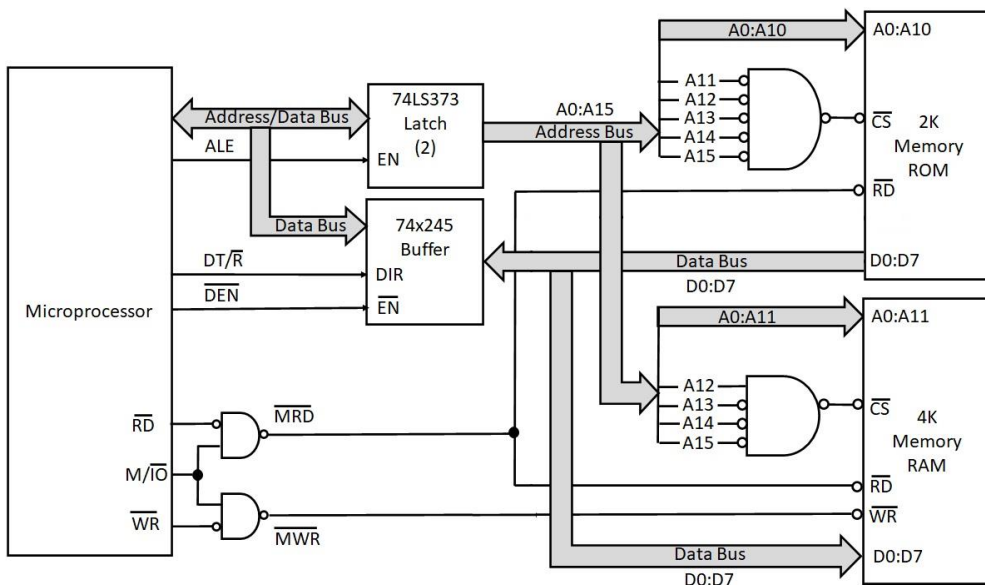


Fig 6.34b Interfacing of 2K ROM & 4K RAM memories – Example 2 & 3

A memory read and memory write cycle is shown in fig 6.34a consisting of 4 cycles each – T1, T2, T3 and T4 states. The ALE is active during T1, Read, write signals enabled during T2 & T3 states. To allow additional time required for slow devices to get ready a wait state T_w is included in timing diagram. When external device is ready to transfer data, it sends an

active high Ready signal to the microprocessor. DEN, RD & WR signals are active during availability of data during T2 to T3.

Example 3: Interface a ROM memory of 2K bytes and RAM memory of 4K bytes with each as a single block of memory and interface 4 I/Os (1 input, 1 output and 2 bi-direction I/Os) using a) memory mapped I/O and b) I/O mapped I/O. Consider 16-bit address & 8-bit data bus.

Sol: The example 2 already shows the interfacing of 2K ROM and 4K RAM. The additional interfacing that is required is for the 4 I/O's in a) memory mapped I/O and b) I/O mapped I/O.

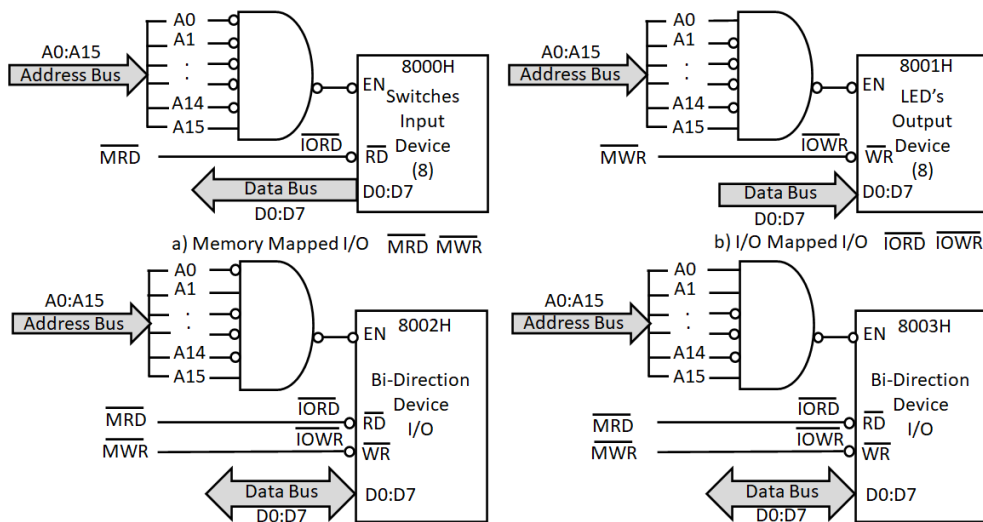


Fig 6.35 Interfacing of I/O's – Example 3

In memory mapped I/O (Von-Neumann type) the ROM, RAM and I/O's share the same address space and should have separate address for each memory, devices & I/O's. Since 0000H to 07FFH is used for 2K ROM, 1000H to 1FFFH are used for 4K RAM, any other alternate address may be used for I/Os. Hence any address between 0800H to 0FFFH or an address above 1FFFH may be used for I/O's. Here 8000H (Input), 8001H (Output), 8002H (Bi-direction) and 8003H (Bi-direction) are used for the 4 I/O's.

The read/write signals for memory or I/O are generated similar to interfacing signals of section 6.6 as shown in fig 6.24. The main interfacing of components of I/O mapped is similar to memory mapped I/O. The main difference lies in the signals used. The memory mapped I/O uses memory read – $\overline{\text{MRD}}$, memory write – $\overline{\text{MWR}}$ signals for I/O's whereas I/O mapped uses IO read – $\overline{\text{IOR}}$, IO write – $\overline{\text{IOW}}$. The interfacing of I/O's is presented in fig 6.35.

Example 4: Interface a ROM memory of 4K bytes and RAM memory of 8K bytes using interleaved odd & even blocks (consecutive bytes in consecutive blocks) of memory using 16-bit address & 16-bit data bus.

Sol:

The example 1 already presented the interfacing of 2K memory and example 2 interfacing of 2K ROM and 4K RAM. However, the memory interleaving of odd and every memory block is required here. The odd block is connected to high data bus D8:D15 and even block is connected to low data bus D0:D7.

2K Byte ROM Allocation → 0000H to 0FFE H Even Addresses

A 15	A 14	A 13	A 12		A 11	A 10	A 9	A 8		A 7	A 6	A 5	A 4		A 3	A 2	A 1	A 0	Address
0	0	0	0		0	0	0	0		0	0	0	0		0	0	0	0	0000H
0	0	0	0		0	0	0	0		0	0	0	0		0	0	1	0	0002H
0	0	0	0		0	0	0	0		0	0	0	0		0	1	0	0	0004H
0	0	0	0		1	1	1	1		1	1	1	1		1	1	1	0	0FFE H

2K Byte ROM Allocation → 0000H to 0FFF H Odd Addresses

A 15	A 14	A 13	A 12		A 11	A 10	A 9	A 8		A 7	A 6	A 5	A 4		A 3	A 2	A 1	A 0	Address
0	0	0	0		0	0	0	0		0	0	0	0		0	0	0	1	0001H
0	0	0	0		0	0	0	0		0	0	0	0		0	0	1	1	0003H
0	0	0	0		0	0	0	0		0	0	0	0		0	1	0	1	0005H
0	0	0	0		1	1	1	1		1	1	1	1		1	1	1	1	0FFF H

Since memory interleaving is required for odd and even the low address bit A0 is used. Hence 4K bytes of ROM is divided into two 2K bytes of ROM with A0 address bit separating odd and even bank. i.e. $4K = 2^{12} = 4096$ bytes.

4K Byte RAM Allocation → 2000H to 3FFE H Even Addresses

A 15	A 14	A 13	A 12		A 11	A 10	A 9	A 8		A 7	A 6	A 5	A 4		A 3	A 2	A 1	A 0	Address
0	0	1	0		0	0	0	0		0	0	0	0		0	0	0	0	2000H
0	0	1	0		0	0	0	0		0	0	0	0		0	0	1	0	2002H
0	0	1	0		0	0	0	0		0	0	0	0		0	1	0	0	2004H
0	0	1	1		1	1	1	1		1	1	1	1		1	1	1	0	3FFE H

4K Byte RAM Allocation → 2000H to 3FFFH Odd Addresses

A 15	A 14	A 13	A 12		A 11	A 10	A 9	A 8		A 7	A 6	A 5	A 4		A 3	A 2	A 1	A 0	Address
0	0	1	0		0	0	0	0		0	0	0	0		0	0	0	1	2001H
0	0	1	0		0	0	0	0		0	0	0	0		0	0	1	1	2003H
0	0	1	0		0	0	0	0		0	0	0	0		0	1	0	1	2005H
0	0	1	1		1	1	1	1		1	1	1	1		1	1	1	1	3FFFH

All the possible 12-bit combinations are required making the address range from 0000H to 0FFFH. i.e. Address bit combination of A1 to A11 with A0 as 1 for odd bank and A1 to A11 address bits with A0 as 0 for even bank. The Addresses 1000H to 1FFFH is not used to reduce chip select complexity similar to example 2. Hence the starting address for RAM used is 2000H.

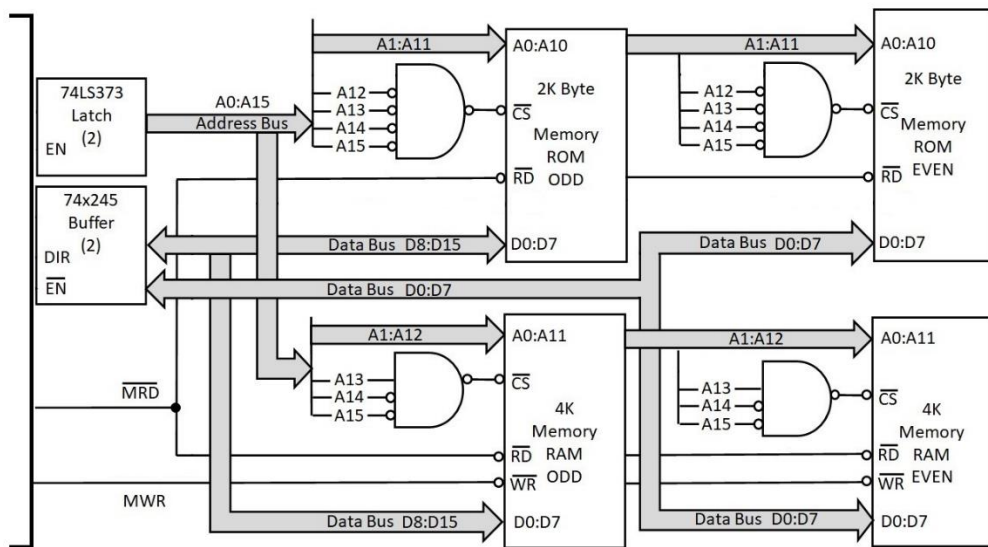


Fig 6.36 a) Interfacing of 4K ROM & 8K RAM - Odd, Even Banks, No A0 – Example 4

The interfacing circuit neglecting the A0 bit is shown in fig 6.36a and the circuit considering the interfacing signal BHE is shown in fig 6.36b.

In cases such as fig 6.36a both the odd and even bank is accessed for a byte or word access as the LSB bit A0 is neglected and PC is incremented by 2 for each access. In PIC18 the program memory neglects LSB bit A0. Hence even if the accessed address is 1001H, it accesses 1000H and 1001H to obtain the 16-bit data. Similarly ARM processors using 32 – bit operations neglect A1 and A0 and require 32-Bit Word alignment. Devices while accessing a byte from one block (as an example - even block) may access complete word but the

processor neglects the other (odd block) data byte and transfers data to the specified byte registers. Similarly, for a 32-bit word, only 16-bit data is considered while other 16-bit data is neglected. In ARM it is achieved through instructions like LDRH for 16-bit access and LDRB for byte access.

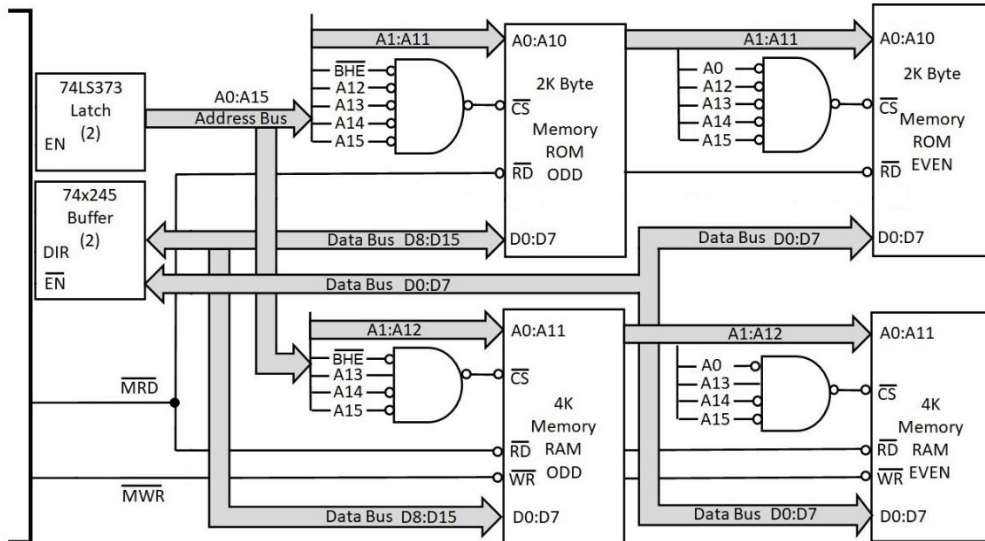


Fig 6.36 b) Interfacing of 4K ROM & 8K RAM - Odd, Even Banks, BHE – Example 4

In cases such as fig 6.36b byte or word access is possible. If the word access required is from odd address, it takes 2 cycles to read 16-bit data as odd address is accessed through high bus first and in the next cycle the even address is accessed through the low bus. This structure is used in 8086.

A Harvard architecture uses separate bus structure and address space for RAM and ROM and hence separate address bus and control signals like ROM write, RAM read, RAM write are separate for the two types of memories.

6.8 Traditional Peripherals

The earliest and famous processors from Intel are the 8085 and 8086. These microprocessors were connected to different components through peripheral interface and used lot of external peripheral devices as they do not have the required in-built peripheral interface or components. The traditional peripherals used are

- Programmable Peripheral Interface – 8255
- Programmable Interrupt Controller – 8259
- Universal Synchronous Asynchronous Receive Transmit – 8251 and
- Direct Memory Access - 8237

These however have been replaced with latest devices and microcontrollers in embedded applications but are presented here for understanding the traditional devices and interfacing.

6.8.1 Programmable Peripheral Interface – 8255

The test 8255 is a programmable peripheral interface that is used to provide parallel I/O capability to microprocessors. It consists of 3 ports PORT A, PORT B & PORT C of 8 I/O lines each making a total of 24 I/O lines. The PORTC is further divided to 4 I/O lines each and called as PORT C Upper and PORT C Lower.

The 24 I/O lines are organized as 2 groups – Group A and Group B. The two groups are comprised of 12 I/O lines each. Group A consists of 8 I/O lines of PORT A and 4 I/O lines of PORTC Upper. Similarly, Group B consists of 8 I/O lines of PORT B and 4 I/O lines of PORTC Lower. The two groups are controlled separately using Group A & Group B control units. The block diagram of 8255 is shown in fig 6.37.

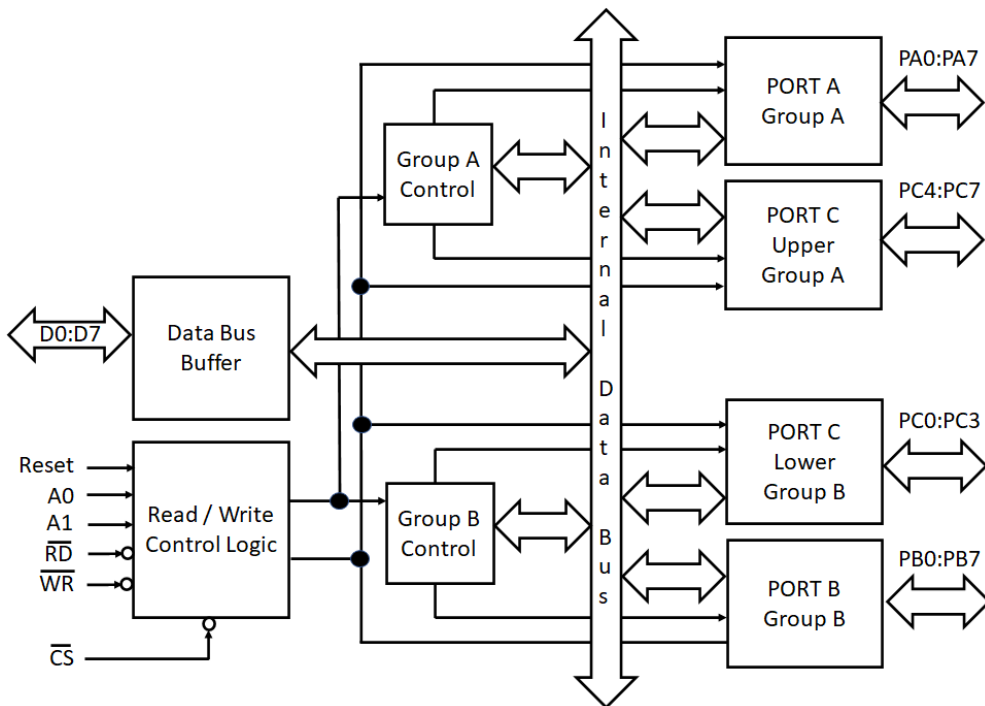


Fig 6.37 Block Diagram of PPI – 8255

The Group A and Group B are configurable to operate in a specific mode. A CWR – Control Word Register is provided for this purpose and is shown below. The modes of Group A and Group B are presented in Table 6.13.

CWR – Control Word Register Format

7	6	5	4	3	2	1	0
M	GAM	GAM	PA	PCU	GBM	PB	PCL
bit 7			bit 0				
Bit No	Bit	Description					
bit 7	M	8255 Mode 1: I/O – Input/Output Mode 0: BSR – Bit Set/Reset Mode					
bit 6:5	GAM1:GAM0	Group A Mode. Refer Table 6.13					
bit 4	PA	Port A I/O					
bit 3	PCU	Port C Upper I/O					
bit 2	GBM	Group B Mode. Refer Table 6.13					
bit 1	PB	Port B I/O					
bit 0	PCL	Port C Lower I/O					

The CWR register allows for configuring 8255 in two modes.

- I/O mode
- Bit Set/Reset Mode.

I/O Mode:

In I/O mode the ports can be configured as input or output. The respective bits of CWR (Port A – PA bit, Port C – PCU bit Upper, Port B – PB bit, Port C Lower – PCL bit) must be set or cleared based on the requirement. The configurable options are given here.

- Set to 0 to configure as output
- Set to 1 to configure as input

Note: The individual bits names given in 8255's CWR register are used to indicate the functionality and are not named specifically by the device manufacturers. These are also not addressable by the given name. The PIC18 registers discussed in this book however have specific bit names given by Microchip and are also addressable by the name.

For I/O mode the CWR is as follows.

7	6	5	4	3	2	1	0
1	x	x	x	x	x	x	x
bit 7				bit 0			

Here x can take a value of 1 or 0 depending on the mode & I/O configuration.

It can be seen from the Table 6.13 that overall, only 3 types of modes exist. They are

- Simple I/O
- Single Handshake I/O
- Bi-Directional Single Handshake I/O – (Port A Only)

Table 6.13: Modes of Group A and Group B in 8255

Group A Mode				Port B Mode	
B6	B5	Mode	Name	B1	Mode
0	0	0	Simple I/O mode	0	0
0	1	1	Single Handshake I/O	1	1
1	0	2	Bi-Directional Single Handshake I/O		
1	1	x	Un implemented		

The operation of Simple I/O is similar to the concept discussed in fig 6.2a of parallel data transfer. The operation of Single handshake I/O is similar to the concept discussed in fig 6.2c and the Bi-directional bus single handshake I/O is similar to that of fig 6.4c. The Port C consists of the required handshake signals. Hence the PORTA and PORTB utilize the PORTC for handshaking signals. Various possible combinations of Group A and Group B modes are presented in table 6.14 where H/S refers to handshake.

Table 6.14: Possible Modes and I/O pins of 8255

Mode		PORT I/O Lines usage				
A	B	A	B	Port C Pins used for H/S		
				A	B	I/O
0	0	8	8	0	0	8
0	1	8	8	0	3	5
1	0	8	8	3	0	5
1	1	8	8	3	3	2
2	0	8	8	5	0	3
2	1	8	8	5	3	0

The handshake signals of PORTA and PORTB as input in single handshake I/O modes are presented in fig 6.38a. The handshake signals of PORTA and PORTB as output in single handshake I/O modes are presented in fig 6.38b. The handshake signals of PORTA as Bi-Directional bus with single handshake I/O mode are presented in fig 6.38c.

The PC0 and PC3 serve as interrupt signals for port B and port A respectively, irrespective of whether the ports are used as either input or output.

Similarly, handshake signals of PORTB remain as PC1 and PC2 but their function change based on input or output configuration. The handshake signals for PORTA in mode 1 varies based on the input or output configuration. When used as input PC4 and PC5 are used as handshaking signals, leaving PC6 and PC7 to be used as I/Os. When used as output PC6 and PC7 are used as handshaking signals, leaving PC4 and PC5 to be used as I/Os.

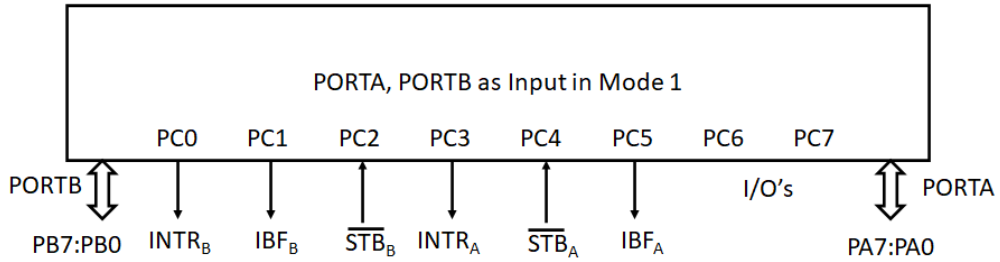


Fig 6.38a Handshake signals of PORTA & PORTB as Inputs in Mode 1

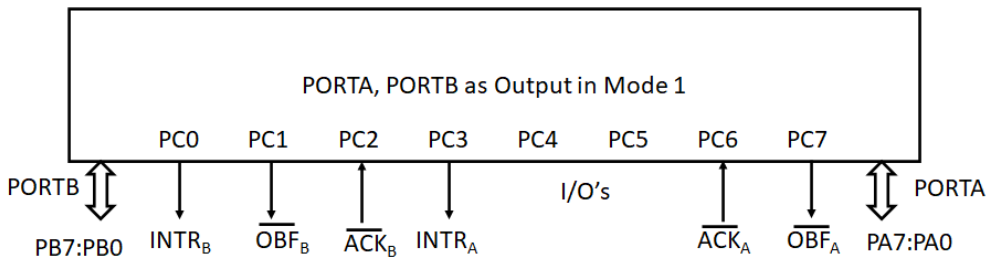


Fig 6.38b Handshake signals of PORTA & PORTB as Outputs in Mode 1

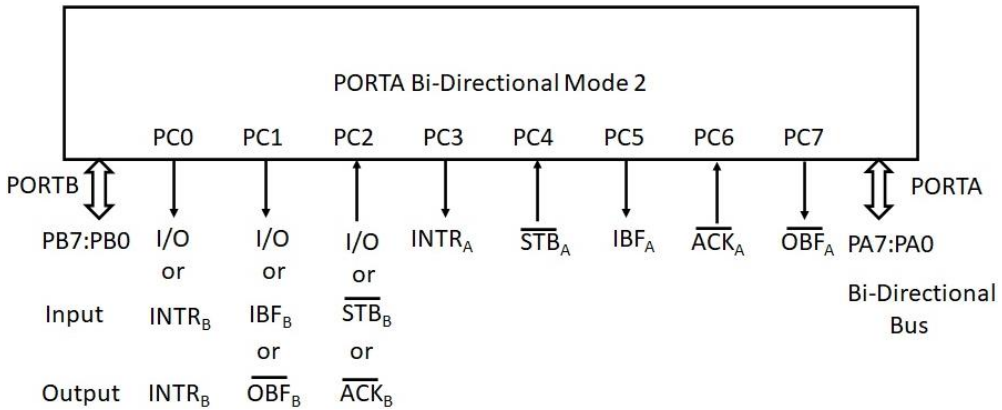


Fig 6.38c Handshake signals of PORTA as Bi-Directional bus in Mode 2

The handshake signal details are equivalent to ones discussed in fig 6.2c. The details are as follows. Here x represents A or B corresponding to PORTA or PORTB.

- **OBF** → Output Buffer Full. It is used in Output mode, Equivalent to Strobe **STB** signal of fig 6.2c. A logic 0 is sent by 8255 to indicate to the receiver that data is on the PORTx for it to read.
- **ACK** → Acknowledge. It is used in Output mode, Equivalent to Acknowledge **ACK** signal of fig 6.2c but in opposite active low state. Receiver acknowledges the 8255's strobe signal that it has received data on PORTx by sending a logic 0.

- STB → Strobe. It is used in Input mode, Equivalent to Strobe STB signal of fig 6.2c. Sent by transmitter to indicate to the 8255 that data is on the PORTx for it to read. A logic 0 on this pin created by transmitter initiates loading PORTx data into the 8255's input latch.
- IBF → Input Buffer Full. It is used in Input mode, Equivalent to Acknowledge ACK signal of fig 6.2c. 8255 acknowledges the transmitters strobe signal about reading of data on PORTx. A logic 1 is sent when data is loaded into 8255's input latch.

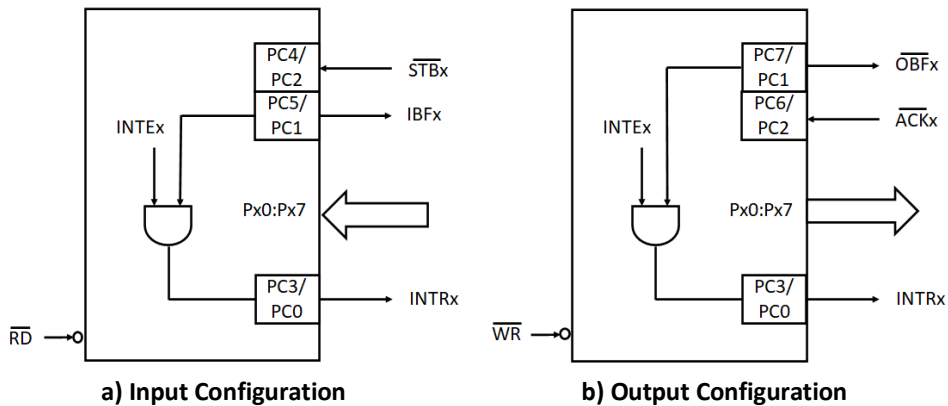


Fig 6.39 Input and Output Configuration - Handshake signals in Mode 1

Note: Numerator indicates Port A's & Denominator indicates Port B's signals

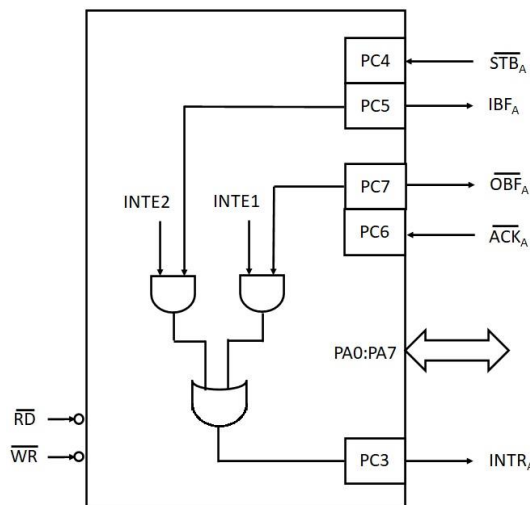


Fig 6.40 Port A Bi-Directional bus - Handshake signals in Mode 2

- INTRx → Interrupt Request.
 - As an input, a logic 1 is generated when STB, IBF and internal flag INTE is set. This interrupts the microprocessor to read the data in the port. The INTE is controlled by bit set/reset of PC4 for PORTA and PC2 for PORTB.

- As an output, a logic 1 is generated when ACK, OBF and internal flag INTE are set. This enables the microprocessor to load new data into port. The INTE is controlled by bit set/reset of PC6 for PORTA and PC2 for PORTB.
- In mode 2 interrupt is generated on PC3 when data is received or sent.

The interrupt generation for mode 1 in input mode is shown in fig 6.39a output mode is shown in fig 6.39b. The mode 2 bi-directional bus signals are shown in fig 6.39c.

Summary of PORTC I/O's,

- When Port A uses single handshake and configured as input in mode 1
 - PC3, PC4, PC5 function as handshake signals.
 - PC6, 7 functions as I/O
 - Use of PC0, PC1 & PC2 as I/O depends on Port B mode, can be used as I/O if Port B is in mode0.
- When Port A uses single handshake and configured as output in mode 1
 - PC3, PC6, PC7 function as handshake signals
 - PC4, 5 functions as I/O
 - Use of PC0, PC1 & PC2 as I/O depends on Port B mode, can be used as I/O if Port B is in mode0.
- When Port A uses single handshake bi-directional bus in mode 2
 - PC3, PC4, PC5, PC6, PC7 function as handshake signals.
 - Use of PC0, PC1 & PC2 as I/O depends on Port B mode, can be used as I/O if Port B is in mode0.
- When Port B uses single handshake either as input or output in mode 1
 - PC0, PC1, PC2 function as handshake signals.
 - Use of PC3, PC4 & PC5, PC6 & PC7 as I/O depends on Port A mode, can be used as I/O if Port A is in mode0. Refer case a, case b and case c for other modes.

Table 6.15: Bit Set/Reset Mode – Selection of PORT C Bit in 8255

S2	S1	S0	Selected Port C Bit	S2	S1	S0	Selected Port C Bit
0	0	0	B0	1	0	0	B4
0	0	1	B1	1	0	1	B5
0	1	0	B2	1	1	0	B6
0	1	1	B3	1	1	1	B7

Bit Set/Reset Mode:

In Bit Set/Reset Mode a specific I/O among the 8 I/O's of PORTC could be selected & could be set or reset. The bit set/reset mode selection of PORTC is shown in table 6.15.

For BSR mode the CWR is as follows.

7	6	5	4	3	2	1	0
0	z	z	z	S2	S1	S0	V
bit 7				bit 0			

Here S2, S1 and S0 are the select bits for selecting the specific PORTC bit and V is the value of I/O bit and can take a value of 1 for set and 0 for reset. The z indicates don't care or unimplemented bits.

Table 6.16: Operations of 8255

<u>RD</u>	<u>WR</u>	<u>CS</u>	<u>A1</u>	<u>A0</u>	Data Transfer
Input (Read) Operation					
0	1	0	0	0	Port A to Data Bus
0	1	0	0	1	Port B to Data Bus
0	1	0	1	0	Port C to Data Bus
0	1	0	1	1	CWR to Data Bus
Output (Write) Operation					
1	0	0	0	0	Data Bus to Port A
1	0	0	0	1	Data Bus to Port B
1	0	0	1	0	Data Bus to Port C
1	0	0	1	1	Data Bus to CWR
Invalid Operation					
x	x	1	x	x	Data Bus Tri-Stated
1	1	0	x	x	Data Bus Tri-Stated

Operation of 8255

The 8255 contains 4 addressable elements. The port A, port B, port C and CWR. They are selected by the address lines A1 and A0. A selection of any one of these four elements enables the chip select and the 8255.

The read or write signals generated by the microprocessor determines the direction of data transfer. A read signal initiates data to be moved from port or CWR to data bus on to microprocessor and a write signal initiates data on the data bus to be written to port or CWR. The data transfers for the respective signal combination are presented in table 6.16. The 8255 must be first configured such that the respective ports are in the required mode and are set as input or output. This is done by loading the corresponding value into the Control word register – CWR. Once configured the 8255 could be used for parallel data transfer on the respective ports.

Example: For PortA, PortB in mode 1, with PortA and PortC lower as input, and PortB and PortC upper as output, the value loaded to CWR is 1011 0101 i.e 95H.

6.8.2 USART – 8251

As seen in the earlier section, microprocessors do not include the parallel or serial port features. However, the parallel and serial port features are available with PIC18 microcontrollers. The parallel port feature for microprocessors is provided through the interface of 8255. The serial port feature is provided through interface of a device such as 8251 – Universal Synchronous Asynchronous Receive Transmit. It provides the asynchronous and synchronous serial communication features and are similar to the different serial data transfer mechanisms presented in section 6.3. The block diagram of 8251 USART is shown in fig 6.41.

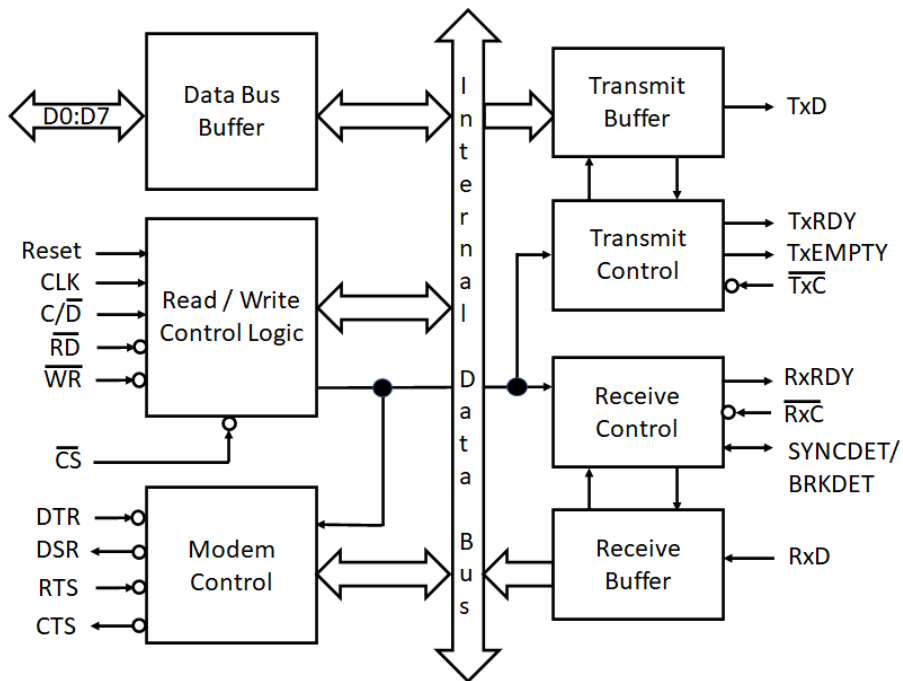


Fig 6.41 Block Diagram of 8251 USART

One of the notable features of 8251 is that it is double buffered as it contains two buffers: 1) Holding buffer and 2) Shifting buffer. While one data is being sent through the shifting register, other data is held using the holding register.

The 8251 is interfaced to system bus of microprocessor through the Reset, clock - CLK, control or data pin- C/D, chip select $\overline{CS}$, read – $\overline{RD}$, write – $\overline{WR}$ signals.

RESET:

The RESET signal is used to reset the 8051 and start from the beginning.

CS: Chip Select:

The chip select signal is generated based on the address assigned to 8251. This is used to enable the 8251.

C/D: Control/Data:

Similar to 8255, the 8251 must be configured before it is used. It also must transfer data to load to transmit buffer for serial transmission and read data from receiver buffer. The C/D signal is used to differentiate and indicate whether the information on data bus is a control word for control register or a data word for data register.

The 8251 makes use of 4 types of words - 1) Mode word, 2) command word, 3) status word along with 4) data word.

Mode Word: The mode word is used during initial setup of 8251 for baud rate, character length, parity and number of stop bit configuration. It is always the first word that must be issued for setup of 8251.

Command Word: It is an operational word which may be used depending on the requirement after the mode word is issued. It is used for enabling the transmitter, receiver, clearing the errors, send break characters, and handshake signals such as DTR and RTS.

Status Word: The status word is used for reading the status of 8251 for obtaining any error information such as parity error, overflow error, frame error, sync detection or the status of Tx_E, TxRDY, RxRDY etc.

Data Word: The data word is the word that is loaded into the transmit buffer or the data read from the receive buffer.

Transmit Buffer:

The Transmit buffer stores the data that must be transmitted serially. The data from data bus of CPU is loaded through the internal data bus of 8251 on to the transmit buffer through a write WR operation signal. The transmit buffer is controlled by the transmit control unit. The data stored in buffer is transmitted serially on the TXD line.

Receive Buffer:

The Receive buffer stores the data that is received serially. The data is then transferred through the internal data bus of 8251 on to the data bus for CPU read RD operation signal. The receive buffer is controlled by the receive control unit. The data sent by the sender is received serially on the RXD line.

Transmit Control:

The transmit control block is enabled by the read read/write control logic unit and is responsible for transmission of data on TXD line according to the selected baud rate in asynchronous communication or the transmitter clock TXC in synchronous communication.

The **TxRDY** - transmitter ready signal is generated by transmit control unit to indicate the CPU that data has been transmitted and that new data could be loaded into the transmit buffer. The TxRDY status may be polled using internal status word. Instead, the

actual TxRDY signal may be used as an interrupt to interrupt the CPU upon the satisfactory condition of TxRDY.

The **TXE** - transmitter empty signal is generated when both the two buffers are empty and has no characters to send. A high-level signal is generated on TXE pin when they are empty, and it is cleared upon receipt of data from CPU. In synchronous communication the TXE signal is used to indicate the end of transmission or that no data is available to be sent. Synch characters are sent when the data is not available.

Receive Control:

The receive control block is enabled by the read/write control logic unit and is responsible for reception of data on RXD line according to the selected baud rate in asynchronous communication or based on the receiver clock RXC in synchronous communication.

The **RxRDY** – receiver ready signal is generated by the receive control unit to indicate the CPU that data has been serially received and that it is available for read by the CPU. The RxRDY status may be polled using internal status word. Instead, the actual RxRDY signal may be used as an interrupt to interrupt the CPU upon the satisfactory condition of RxRDY.

The **SYNDET/BD** – Synch Detect and Break Detect pin may be used as an input or output. After device reset it is configured as output and is reset after a status read. In asynchronous mode it acts as break detect or line break, indicating break in transmission. It is detected when RXD line has low value in two or more-character bit periods. In synchronous mode it is used to detect synch characters. These are required to ensure that the receiver is synchronized w.r.t the sender's clock. As an output pin it indicates detection of synch characters on RXD line. As an input pin, a positive edge causes 8255 to start assembling data characters from the next rising edge on RXC clock.

The handshake signals DTR, DSR, RTS, CTS signals and its functions is similar to the ones discussed in RS232 standard of section 6.4 and the asynchronous and synchronous communication formats have been presented in the section 6.3 of serial data transfer and hence is not discussed here separately. Interested readers are encouraged to read the data sheet of 8251 for further details. The USART features of PIC18 are presented in detail in ch.10.

6.8.3 Programmable Interrupt Controller – 8259

Microprocessors or microcontrollers include only a limited number of physical pins for interrupts on its IC package to ensure a minimum IC size. Due to this limited number of interrupt related pins, very few interrupt sources could be connected. The interfacing shown

in Fig 6.26 during the discussion of interface pins INTR & INTA demonstrate an example of Sensor/Actuators interfacing using interrupts.

In cases where numerous interrupt sources are present, a device such as the 8259 – Programmable Interrupt Controller is used. One 8259 extends the interrupt source capability from 1 to 8. The interrupt sources can be extended further when 8259s are cascaded to form a hierarchical structure of nine 8259's. In a hierarchical cascade structure one 8259 acts as a master and is connected to microprocessor while the other 8 act as slaves to the 8259 masters. This allows for increasing the interrupt source capability from 1 to 64. It can be further extended when the number of levels & devices is increased further. The hierarchical or cascading structure is demonstrated at the end of this section.

The block diagram of 8259 is shown in fig 6.42 and is described as follows.

IR0:IR7 – Interrupt Sources

The 8259 can be sourced by 8 interrupt sources and must be connected to the respective input lines named as IR0:IR7.

Interrupt Request Register – IRR

The purpose of Interrupt request register - IRR is to maintain the status of the 8 interrupt requests on IR0:IR7. The respective interrupts are serviced later by the 8259 based on the interrupt's priorities.

Interrupt mask register – IMR

Any of the 8 interrupts can be masked/disabled or unmasked/enabled by setting or resetting the corresponding bit of the Interrupt mask register.

Priority Resolver

A priority number exists for all the interrupts. The priority number of an interrupt may vary based on whether it is a fixed priority or a rotating priority. The priority resolver ensures that the interrupts are serviced in the order of higher priority to lower priority. The highest priority interrupt is selected and the corresponding bit in the Interrupt Service Register - ISR is set upon an interrupt acknowledge INTA pulse and the corresponding bit in Interrupt Request Register IRR is reset.

In Service Register – ISR

The interrupt Service Register - ISR holds the status of interrupts that are currently being serviced. The service routine of more than one interrupt may be in the execution process. This happens when the execution of low priority interrupts interrupt service routine is interrupted by several higher priority interrupts.

Interrupt control logic

It is the interface and control between CPU and 8259. It generates an INTR signal in response to an active interrupt that is set in IRR. It processes the interrupt acknowledge signal INTA received from CPU, set the corresponding ISR bit and clears the equivalent IRR bit. This enables the 8259 to place the interrupt vector address on the data bus when a second interrupt acknowledge signal INTA is received.

Data Bus Buffer

It is an 8-bit tri-state bi-directional buffer. It is used to interface with data bus of microprocessor through D0:D7 data lines.

Read/Write Control Logic

It is interfaced with CPU through read, write signals, one address bit and a chip select signal. The 8259 is initialized and operated using word values known as initialization command words – ICWR1, ICWR2, ICWR3 & ICWR4 and operation command words – OCWR1, OCWR2 & OCWR3. The read and write signals are used to perform read or write operations of these initialization or operational command words and registers. So, it accepts codes and decodes commands from CPU. The address bit A0 determines whether the value sent is an ICWR1 or any other word. The A0 is 0 for ICWR1 and 1 for the other ICWR words and is 1 for OCW1 and 0 for OCW2 and OCW3.

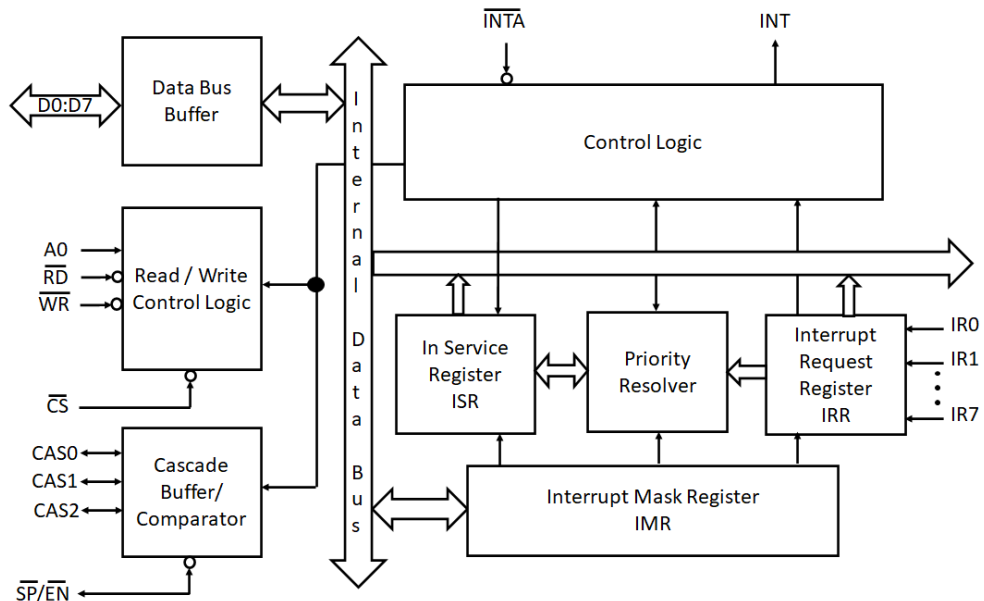


Fig 6.42 Block Diagram of 8259

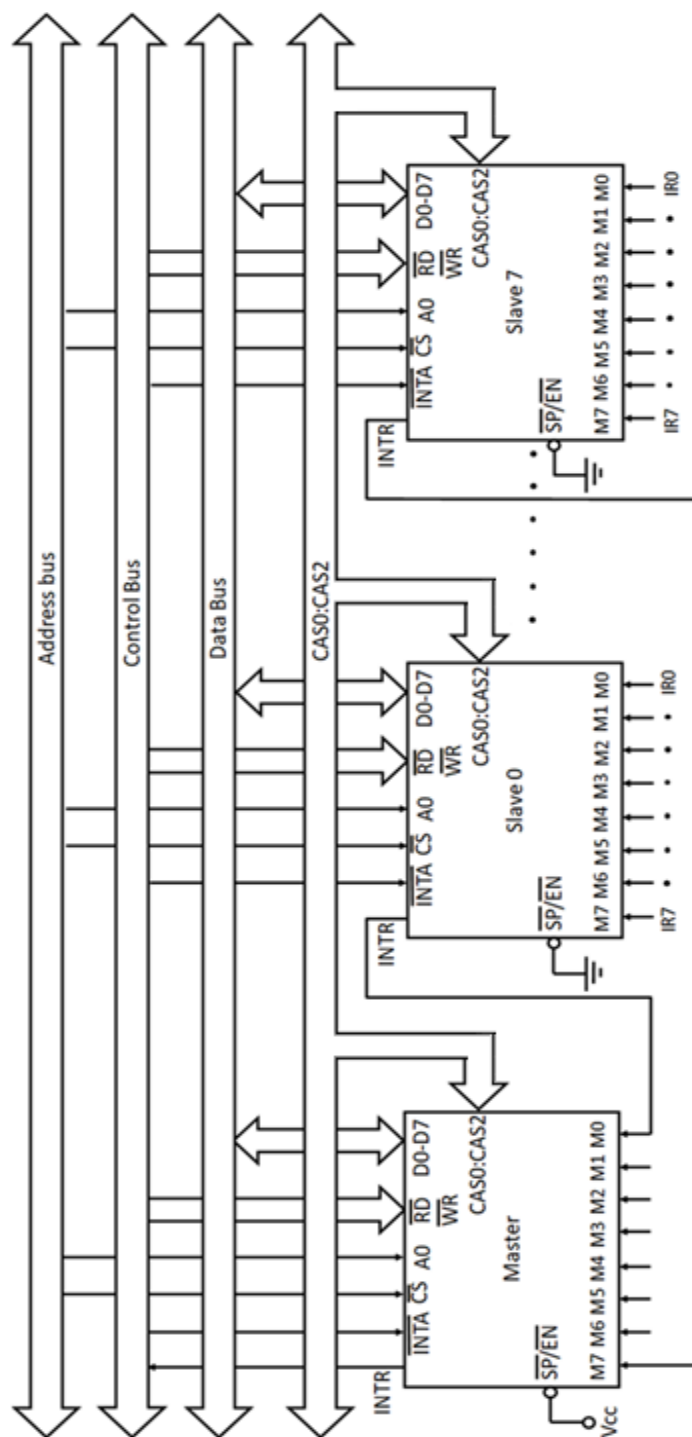


Fig 6.43 Cascading of 8259's

Cascade Buffer/Comparator

It is mainly used in cascaded 8259 operations. It stores and compares all IDs of all 8259A's in the system. When more than 8 interrupts are required, cascading of 8259s is required. So, an 8259 can be a master or a slave. Each slave has an ID. This module provides cascade lines CAS0:CAS2. As a master the CAS0:CAS2 lines provide output signals to select or identify one of the slaves. As a slave, the CAS0:CAS2 lines provide input lines for reading. The master provides ID on these lines, and the slave compares the value with its ID and if a match occurs it responds with the vector address on data bus upon next INTA pulse.

The SP/EN pin acts as an output buffer enables to control buffer transceivers in buffered mode (EN). It is otherwise an input pin used to indicate whether the device is a master or slave. When the 8259 is used as a master SP pin is to be tied to VCC else if it is a slave, it must be connected to ground. The cascading arrangement of 8259 is shown in fig 6.43. Details description of the formats of ICWR's and OCWR's is not presented here and interested readers are advised to refer to the data sheet for their details.

Operation Sequence

- After initialization of 8259, any one or more of the 8 interrupt sources on IR0:IR7 may set the corresponding IRR bits.
- Priority resolver finalizes the highest priority interrupt request.
- Control Logic generates an INTR request to CPU.
- CPU/microprocessor acknowledges through INTA pulse.
- After the INTA from CPU, the highest priority interrupt related bit in ISR is set and respective IRR bit is reset.
- If in cascade mode, the master 8259 sends the slave id on CAS0:CAS2.
- CPU/microprocessor sends 2nd acknowledge INTA pulse.
- If only one 8259 is present, the single 8259 places vector corresponding to interrupt request IR0:IR7.
- If in cascade mode the slave whose ID matches to that of ID sent by master, places the corresponding interrupt vector data corresponding to the interrupt on the data bus.
- CPU reads the vector location and starts the corresponding service routine.
- The respective ISR bit is cleared if 8259 is in auto end of interrupt mode upon the 2nd INTA pulse. It remains set in other modes till the completion of the interrupt and till the issuance of end of interrupt command at the end of service routine.

6.8.4 Direct Memory Access

A microprocessor-based system is designed to perform various tasks such as acquiring different sensor signals, storing the data to memory, later processing of the signal data and displaying the required information or activate some other devices etc. Transferring data between memory and I/O devices is required in data intensive applications such as signal and image processing where continuous stream of data is generated. The data transfer is typically performed by the microprocessor or a microcontroller when there is no provision for direct transfer of data or when a separate device is not present for such a purpose. When the same processor is used for performing all the tasks, it can either perform the task of transferring the data between memory and I/O or the other required tasks at any point of time. When it is used for data transfers, the microprocessor is diverted from doing the other required tasks. This limits the ability of the designed system and may not be useful for time critical applications. For an efficient use and to perform the given tasks in time critical applications, a device such as a direct memory access – DMA may be used.

6.8.4.1 Interfacing Direct Memory Access

The DMA – Direct memory access is a programmable device that performs the task of transferring the data between the memory and I/O without the intervention of the processor. This allows the microprocessor to perform the required tasks while the DMA transfers the data between memory and IO. A classic and commonly used DMA is the 8237. It is discussed later in this section.

A generalized block diagram of microprocessor-based system that employs DMA is presented in fig 6.44. Various signals (such as HOLD, HLDA etc.) that are used for interfacing of such a system have been presented in the earlier section of signal interface in section 6.6 of this chapter. The operational steps of a generalised DMA based system are as follows.

- Upon powering ON the microprocessor, execution of program instructions proceeds. During this period the systems bus consisting of address bus, data bus and control bus is in the control of microprocessor and bus outputs of DMA are tri-stated.
- An I/O connected to a DMA generates a DMA request DREQx.
- In response to I/O devices request DREQx, the DMA generates an active high HOLD signal to the microprocessor.
- In response to the DMA's HOLD request, the microprocessor completes the current instruction execution and releases control of the system bus when no other locking condition exists.

- It then sends an active high hold acknowledge signal – HLDA to the DMA indicating it has released the system bus & that the DMA can become master of it.
- Now the DMA is the master and controls the system bus which in turn generates a DMA acknowledge signal to the requesting I/O device indicating that the system bus is under its control and data transfer can commence as shown by switch settings connected by system bus to memory from the DMA in fig 6.44.
- The DMA, memory and I/O device may then communicate by control and status signals and perform the task of transferring the data between memory and I/O without the intervention of the CPU. The DMA generates the memory read/write or IO read/write signals to transfer data between memory and IO.
- The CPU may perform any other tasks and execute operations that do not require the system bus, memory or I/O's. The 8086 has 6-byte instruction queue which could store up to 6 instructions. Similarly, some microprocessors have an instruction cache of few MBs which may be loaded with prefetched instructions. So, these prefetched instructions may be executed if they do not require access to memory, I/O etc while memory & I/O data transfer takes place.

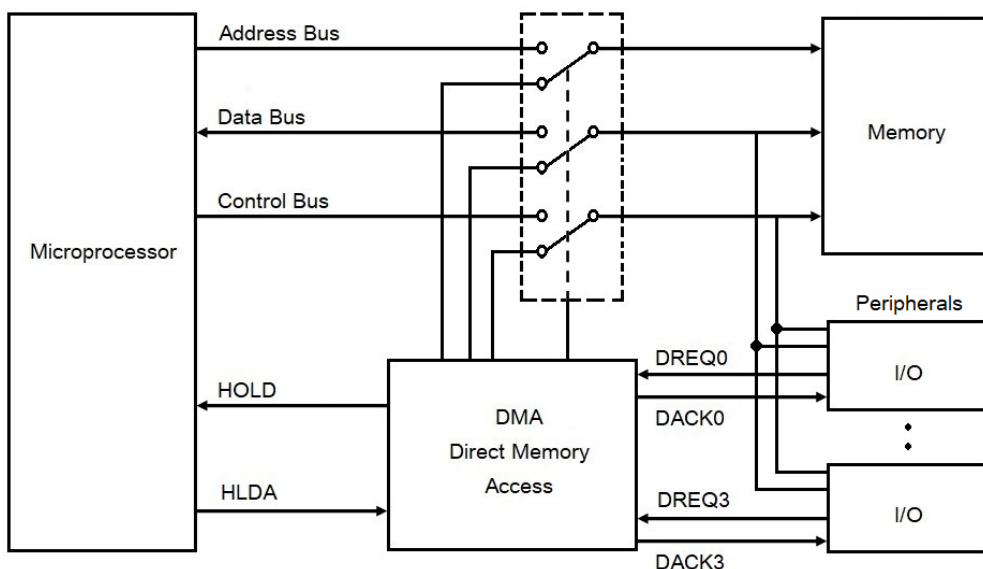


Fig 6.44 Typical Microprocessor & DMA interconnection system

6.8.4.2 DMA 8237

There are two classical DMAs, 8257 and 8237. The features of 8237 DMA are

- Four DMA Channels with four separate DMA request I/P (DREQ0:DREQ4) and acknowledge O/P signals (DACK0:DACK3).

- Each channel provided with base address, base word count, current address and current word count registers of 16-bit each.
- Status register to indicate channel request status and finished (TC reached) status
- Common temporary address, word count and data register.
- Four address select lines are provided to select among the various internal registers.
- Supports data transfers between memory and I/O as well as memory to memory transfers.
- 8237 DMA cascade feature for over 4 data transfer request types.
- Software programmable/Configurable Features
 - Programmable increment or decrement address operations
 - Programmable Request Register – Set/Reset of requests
 - Auto initialize/reload of address and count registers
 - Selectable transfer modes - single, block and demand mode
 - Individual or group mask control bits
 - Normal/compressing timing
 - Fixed or rotating priority
 - Verify/Write/Read
 - Late/extended writes
 - Programmable DMA request detection based on active high or low level

Another popularly used DMA is the 8257. Some of the features that are present in 8237 but are not present in 8257 are as follows. 1) Cascading feature 2) memory to memory transfer 3) Increment/decrement configurability, 4) 4 separate mode registers 5) 16-bit count registers, has only 14-bit register etc.

The various registers of 8237 are presented in table 6.17 and the block diagram is shown in fig 6.45.

Base Address Register:

The base address register is used to hold the starting memory address of the corresponding channel. It is a 16-bit register and a separate register exists for every channel. The value of this register is normally held constant. During auto initialization the value of this register is loaded into the current address register.

Table 6.17: Internal registers of 8237

Register Name	Size (bits)	Number of Registers
Base Address Registers	16	4
Base Word Count Registers	16	4
Current Address Registers	16	4
Current Word Count Registers	16	4

Temporary Address Register	16	1
Temporary Word Count Register	16	1
Status Register	8	1
Command Register	8	1
Temporary Register	8	1
Mode Registers	6	4
Mask Register	4	1
Request Register	4	1

Base Word Count Register:

The base word count register is used to hold the count value of the number of required data transfers. It is a 16-bit register and a separate register exists for every channel. The value of this register is held constant. During auto initialization the value of this register is loaded into the current word count register.

Current Address Register:

The current address register holds the address of the memory currently being accessed. This address is modified after every data transfer based on the choice of increment or decrement. The current address register is a 16-bit register and a separate register exists for every channel. The current address register value varies with the number of data transfers. This register is reloaded with value from base address register if auto initialization is selected.

Current Word Count Register:

The current word count register holds the value of the unfinished data transfers (i.e number of data transfers that are yet to be completed). The current word count register is a 16-bit register and a separate register exists for every channel. The count value is decremented after every data transfer. The current word count register value varies with the number of data transfers. This register is reloaded with value from base address register if auto initialization is selected.

Temporary Address and Count Register:

The current address and current word count register values are copied to the temporary address and temporary word count registers for performing increment/decrement operations. The values of the temporary registers are placed on address and data bus.

Temporary Data Register:

The 8237 allows data transfers between memory to memory in addition to the usual transfers between memory and IO. The memory-to-memory transfer is made possible by having an 8-bit temporary data register that is provided to hold the temporary data for transfer between two memories. It is implemented in two cycles. During the first cycle, a memory read operation is performed, where memory1 is read & the respective data is stored

in temporary register, and during the second cycle, the temporary register is read and the respective data is stored into the memory2 using a write operation.

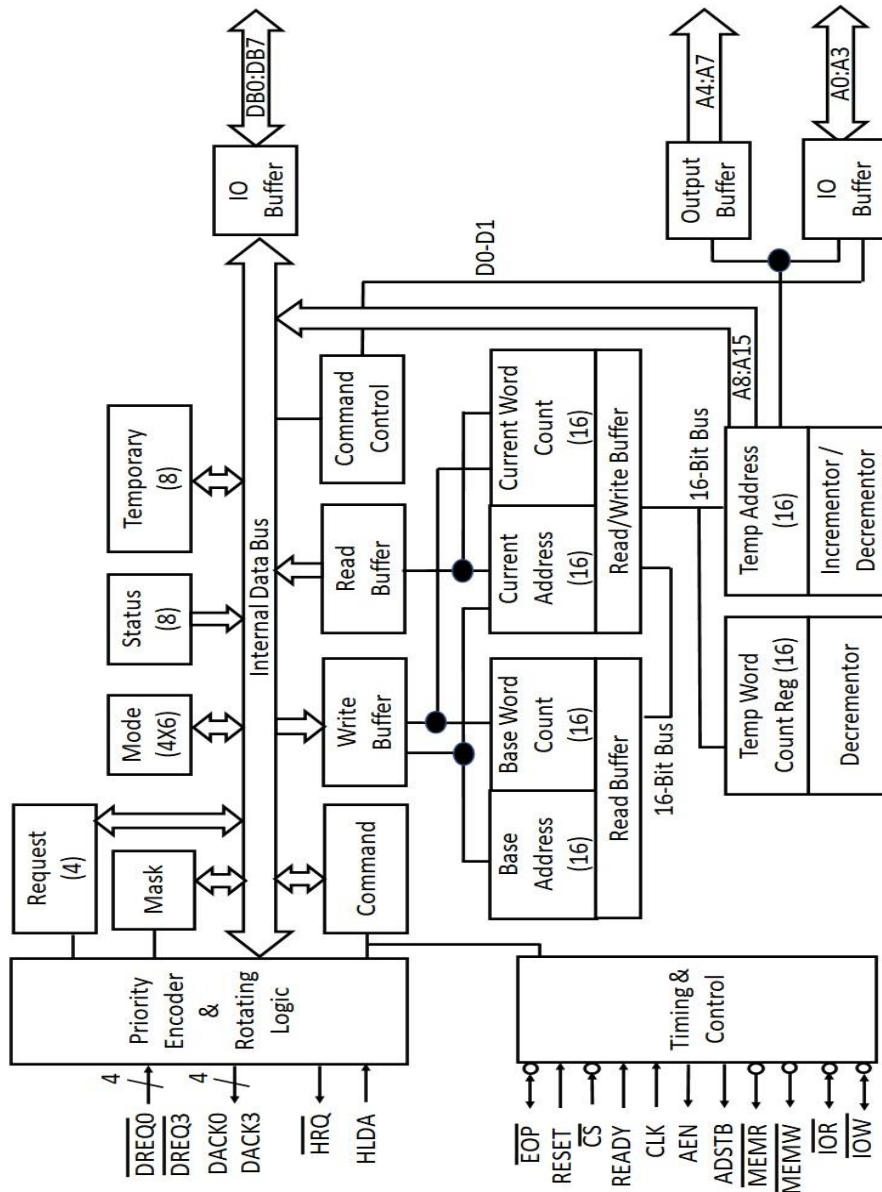


Fig 6.45 Block Diagram of 8237 DMA

Mode Register:

The 4 channels of 8237A DMA are each provided with a 6-bit mode register. The lower two bits of the mode word are used to select the respective channel. The other 4 bits of mode register are used to select increment or decrement of address, auto initialize enable,

type of select mode – (Demand, Single, Block or cascade mode) or transfer type (Verify, Write, Read). Since each channel is provided a separate mode register, a channel can be selected for read operation and other channel for a write operation. Similarly, a channel could be used in single transfer mode while other channel may be used in block transfer mode or cascade mode etc. In addition to this one channel could access memory from lower to higher address (Increment) while the others could access from higher to lower address (Decrement) and auto reloading of current registers from base registers could be performed through the Auto-initialization enable option. Auto-initialization also allows next set of transfer upon a DMA request without intervention of CPU.

Transfer Types: The write transfer type is used to transfer data from I/O for it to be written to memory. The read transfer type is used to transfer data from memory to I/O using a memory read operation. Verify transfers are pseudo transfers similar to read or write with the exception that memory and I/O lines are deactivated.

Transfer Modes: In single transfer mode only one transfer is made each DMA request. In block transfer, multiple transfers (equivalent to count) are made for every DMA request. In demand transfer, data transfers are repeated until an external signal is received. When an active low external signal is applied on End of Process - EOP pin the DMA transfers are terminated. When more than four DMA requests are needed, the DMAs must be cascaded as shown in fig 6.46. When a DMA request input is cascaded, the corresponding channel must be configured to be in cascade select mode.

Command Register:

The 8237A contains only one 8-bit command register to control the operation of DMA. This register is used to enable or disable memory to memory transfers, channel 0 address hold, controller features. It also allows selecting normal/compressed timing, fixed/rotating priority, late/extended write, active high/low DREQ and DACK detection.

When memory to memory transfer is selected by setting bit 0 in command register, channels 0 and 1 are selected for memory-to-memory transfers. The setting of DREQ0 by software setting initiates the transfer operation. Channel 0 registers are used to read memory and channel 1 for write to memory. The data read from memory using channel 0 is stored in temporary register, which is then written to memory using channel 1.

In fixed priority the priorities remain fixed as DREQ0 being high priority and DREQ3 the low priority. In a rotating priority the channel that is recently serviced is made low priority while the channel that was serviced the oldest among all the channels is made the highest priority. This priority is rotated continuously. The extended write is used for slower devices or a higher time lag. Similarly, compressed timing is used for faster devices to increase access speed.

Timing & Control Unit:

This unit is responsible for generation of internal timing and external control signals. It operates based on clock input from microprocessor or separate clock source.

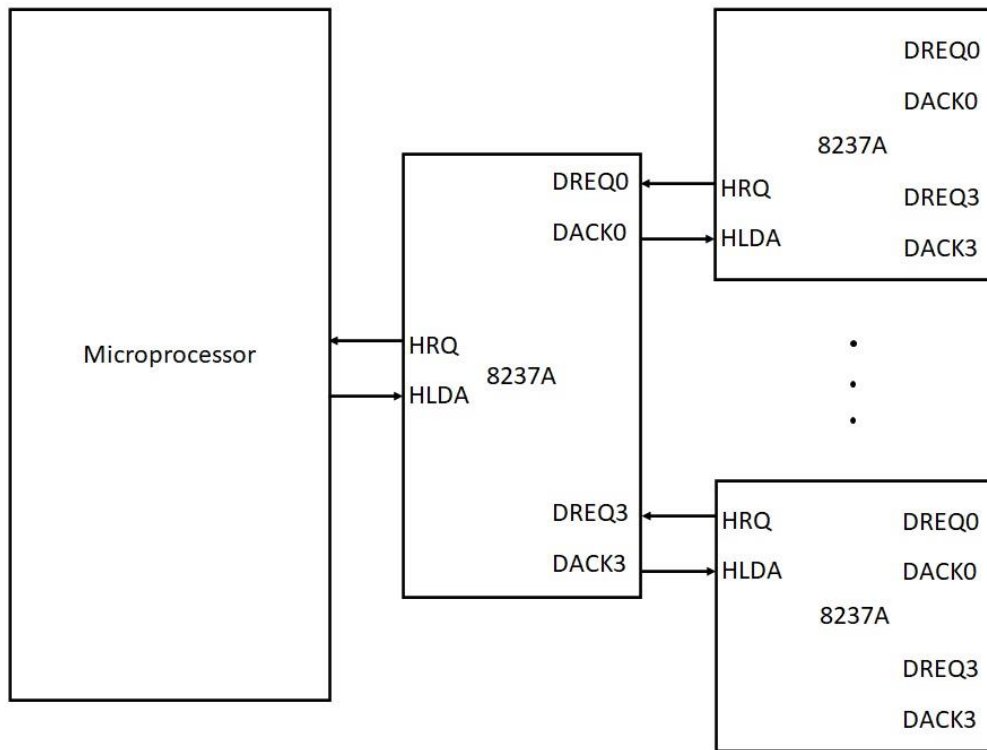


Fig 6.46 Block Diagram of cascading 8237 DMA's

Request Register:

The DMA transfer can be caused by an actual DMA request at DREQ input pin or software by setting the request for the corresponding channel. The software selection format has lower two bits B1, B0 to select the channel and the B2 bit is set/reset to create or clear a DMA request operation. The request register is a 4-bit register, with LSB corresponding to channel 0 and 3rd bit corresponding to channel 3. The software command with a "110" sets DREQ2 i.e. DMA request of channel 2. It sets the 2nd bit of the request register (in 0th to 3rd bit notation). The software setting of DMA requests are non-maskable but are processed through the priority encoder and rotating logic. The registers DREQx bit is cleared upon completion of respective data transfers i.e. upon a TC or external EOP.

Mask Register:

The DMA requests DREQx are masked by setting the corresponding bit to a 1. It is done similar to software set of DMA request individually using separate commands or all the masks may be set using a single command. Each DREQx mask is set upon completion of DMA

transfer operations of a channel. All masks are set, DREQs disabled upon device reset. A mask clear command either individually or a group is required before a use.

Status Register:

The status register is an 8-bit register and contains information about whether any channels DMA request DREQx is active i.e. bit set in request register, and if DMA transfer operations are completed. It contains status of all the 4 channels DMA requests and TC reached status. The lower 4 bits indicates TC reached status and is set to 1 to indicate completion of channels data transfers. The higher 4 bits indicates the pending DREQx requests that are yet to be completed.

Users are encouraged to refer the datasheet of the 8237A for the register formats, register selection bit combination and for further details.

Pin Description:

Here only the pins that are specific to the DMA are discussed and common signals such as power, clock chip select etc are left to the reader's discretion.

READY: The ready signal is used to check the readiness of the memory or I/Os. When the memory or peripheral is not ready to accept, read or write data it makes the ready pin low. The high on this pin indicates readiness of memory or peripheral for data transfers.

HRQ, HLDA – Hold, Hold Acknowledge: These are similar to HOLD & HLDA discussed in interfacing signals section 6.6.

DREQ0:DREQ3 – DMA Request: These are 4 DMA channel requests available on the DMA for use by peripherals to initiate data transfers. DREQ0 is highest priority while the DREQ3 is the lowest.

DACK0:DACK3 – DMA Acknowledge: These are 4 DMA channels acknowledgement signals generated in response to DMA requests DREQ0:DREQ3 upon the receipt of a hold acknowledge – HLDA from processor. This indicates DMA transfer operation is granted. The acknowledge signal is first sent to peripheral connected to high priority channel.

IOR, IOW – IO Read, IO Write: These are bi-directional 3-state pins. These pins act as input pins when DMA is a slave while the microprocessor is the master controlling the system bus. It is used to enable read or write of DMA's registers. These pins act as output when DMA is the master of the system bus after a HLDA signal from microprocessor. As output these are used to read or write a peripheral.

MEMR, MEMW – Memory Read, Memory Write: These are 3-state output pins used in memory read and memory write operations respectively.

A0:A3 – Address Bus: These are bidirectional 3-state I/O lines. When DMA is the slave, they are inputs and are used to select the internal registers for reading and writing address, count, command, mode and status words. When DMA is master, they are outputs and provide low 4-bits of memory address

A4:A7 – Address Bus: These are 3-state outputs when DMA is master and provide high 4-bits of low 8-bit memory address.

D0:DD7 – Data Bus: These are bidirectional 3 state lines. When the CPU is the master, the data to and from CPU is transferred on these lines. When DMA is the master, these D0:D7 lines act as multiplexed address/data lines. During the first cycle they act as high 8-bit address A8-A15 and later are used for transferring of data for memory-to-memory transfers.

ADSTB – Address Strobe: When DMA is the master and high 8-bit address A8:A15 is generated on the D0:D7 lines, active high ADSTB signal is generated by DMA and must be used as strobe for storing the upper address into the latch.

AEN – Address Enable: The active high address enable signal is used to enable the latch containing the high 8-bit address to place latch's data on to the system address bus and disable the other system bus drivers when DMA is the master.

EOP – End of Process: The EOP is an active low bidirectional signal. In demand transfer mode this pin is used as an external input for terminating the active DMA data transfers. An active low signal must be connected to terminate the transfers. This is an active low output in other transfer modes and indicates the completion of transfer's i.e. a pulse is generated when Terminal Count for any channel is reached. When terminal count is reached the respective mask bit is set if it is not configured for auto-initialization. The mask bit remains unchanged if a channel is set for Auto-initialization.